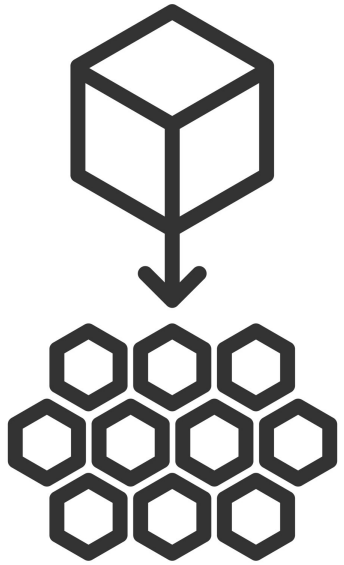
Running and Securing Kubernetes Apps

TEKsystems Global Services

WORKFORCE DEVELOPMENT

Microservices evolution

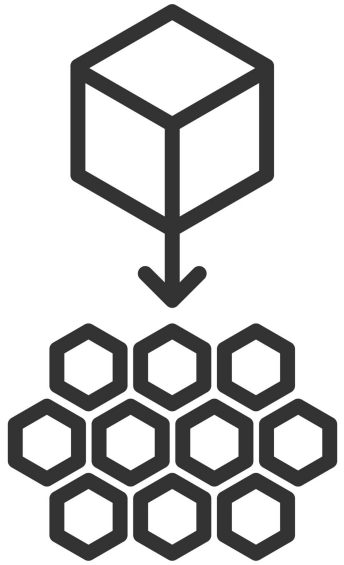
Microservices: the modern application model



Microservice Architecture

Companies with large, monolithic applications are increasingly breaking these unwieldy apps into smaller, containerized microservices.
Microservices are popular because they offer agility, speed, and flexibility. They allow teams to build, deploy, and scale features independently. However, they also introduce complexity that can become a hurdle for adoption.

The evolution of application development

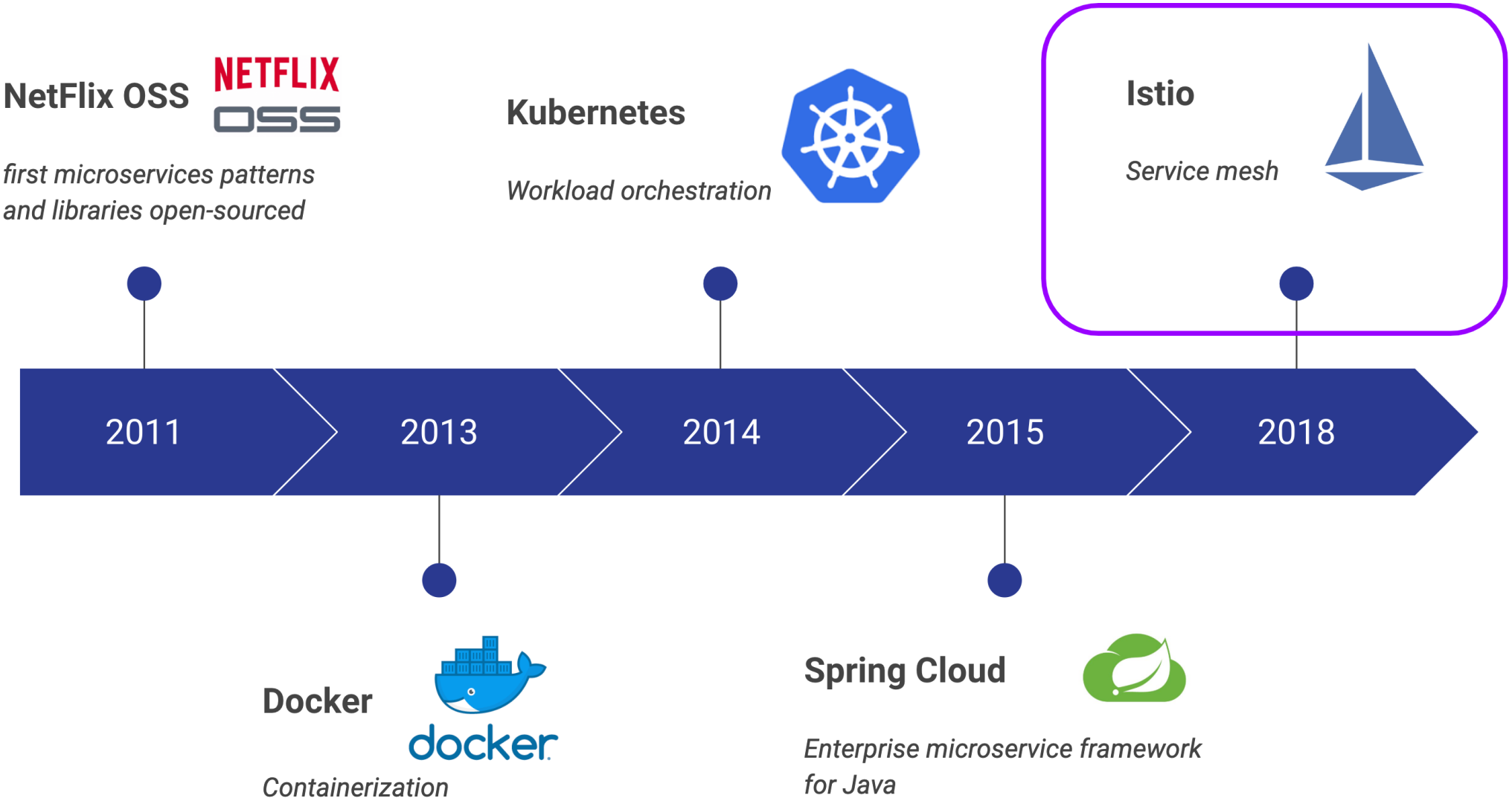


Microservice Architecture

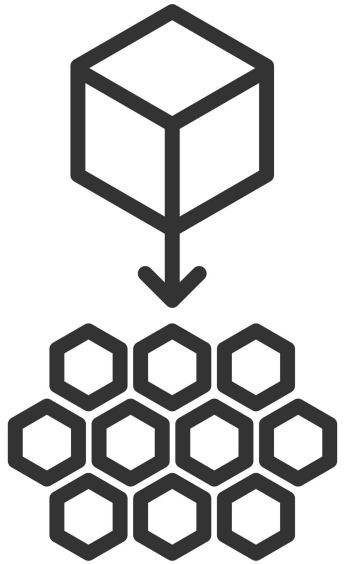
Software development practices have fundamentally changed the way applications are designed and delivered.
Applications have transformed from large monolithic code bases to collections of many small, loosely coupled services.
This shift is driven by the desire to ship software faster and in a way that is portable across diverse infrastructure environments.

- Enables agile development and rapid iteration
- Improves scalability by allowing individual components to grow independently
- Supports flexibility in language and technology choices

The evolution of application development



Microservices challenges



Microservice Architecture

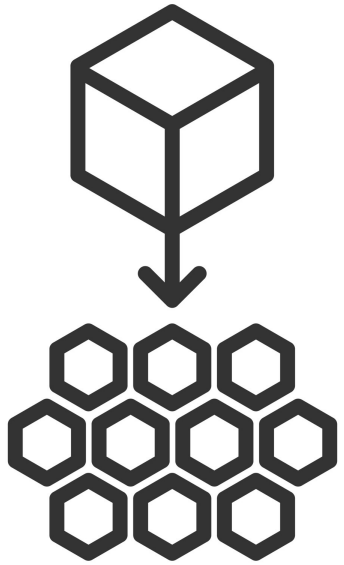
Adopting a microservices-oriented architecture brings new challenges:

- M to N communication.
- Distributed software interconnection and troubleshooting are complex.
- Containers should remain thin and platform-agnostic.
- Upgrading polyglot microservices is challenging at scale.

Microservices challenges II

- *Configuration Service*
- *Service Registry / Discovery*
- *Circuit Breaker / Retry*
- *Rate Limiting*
- *Event Driven Messaging (Async)*
- *Audit*
- *Load Balancing / Intelligent Routing*
- *API Gateway*
- *Authentication & Authorization*
- *Monitoring*
- *Distributed tracing*
- *Log Aggregation*

Business value and microservice concerns

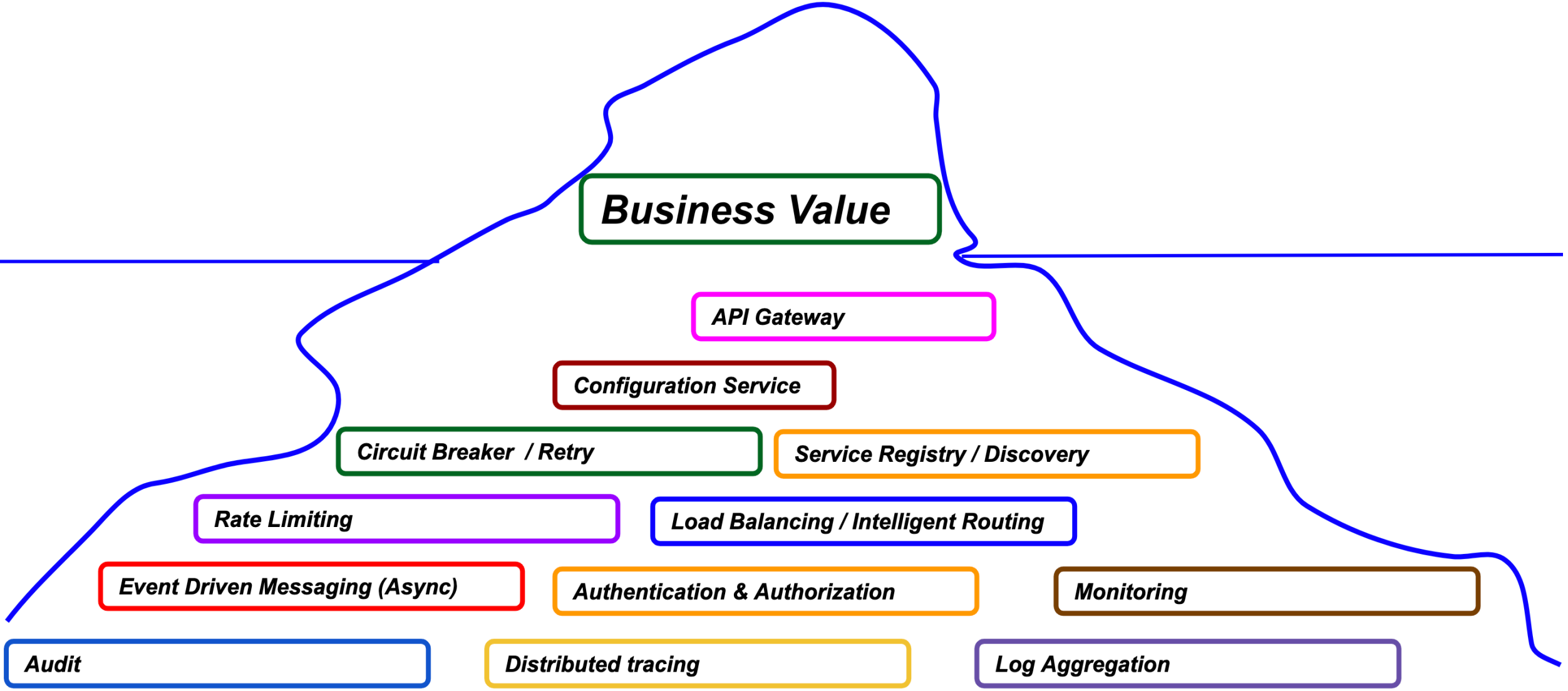


Microservice Architecture

To business stakeholders, the only visible outcome of technology is the business service itself.
Delivering new value in a microservices world means focusing on the key concerns that make services reliable, scalable, and ready to support the business.

- Service boundaries aligned with business capabilities
- Stable APIs that support independent development
- Data ownership and clear consistency rules
- Security and compliance built in from the start
- Observability for monitoring and troubleshooting
- Scalability and resilience to handle change and failure
- Automated delivery with CI/CD and orchestration

Business value and microservice concerns



What is a service mesh?

Building the target microservice platform

As mentioned earlier, building distributed and disparate microservices requires a cohesive platform.

The platform provides the technical foundation to support microservices at scale.

- Service discovery & networking to connect and route between services
- Runtime orchestration with containers and automated scaling
- Security & compliance built into the platform
- Observability & monitoring for logs, traces, and metrics

Services team focus

With the platform handling the technical concerns, the services team can focus on what the business sees and values.

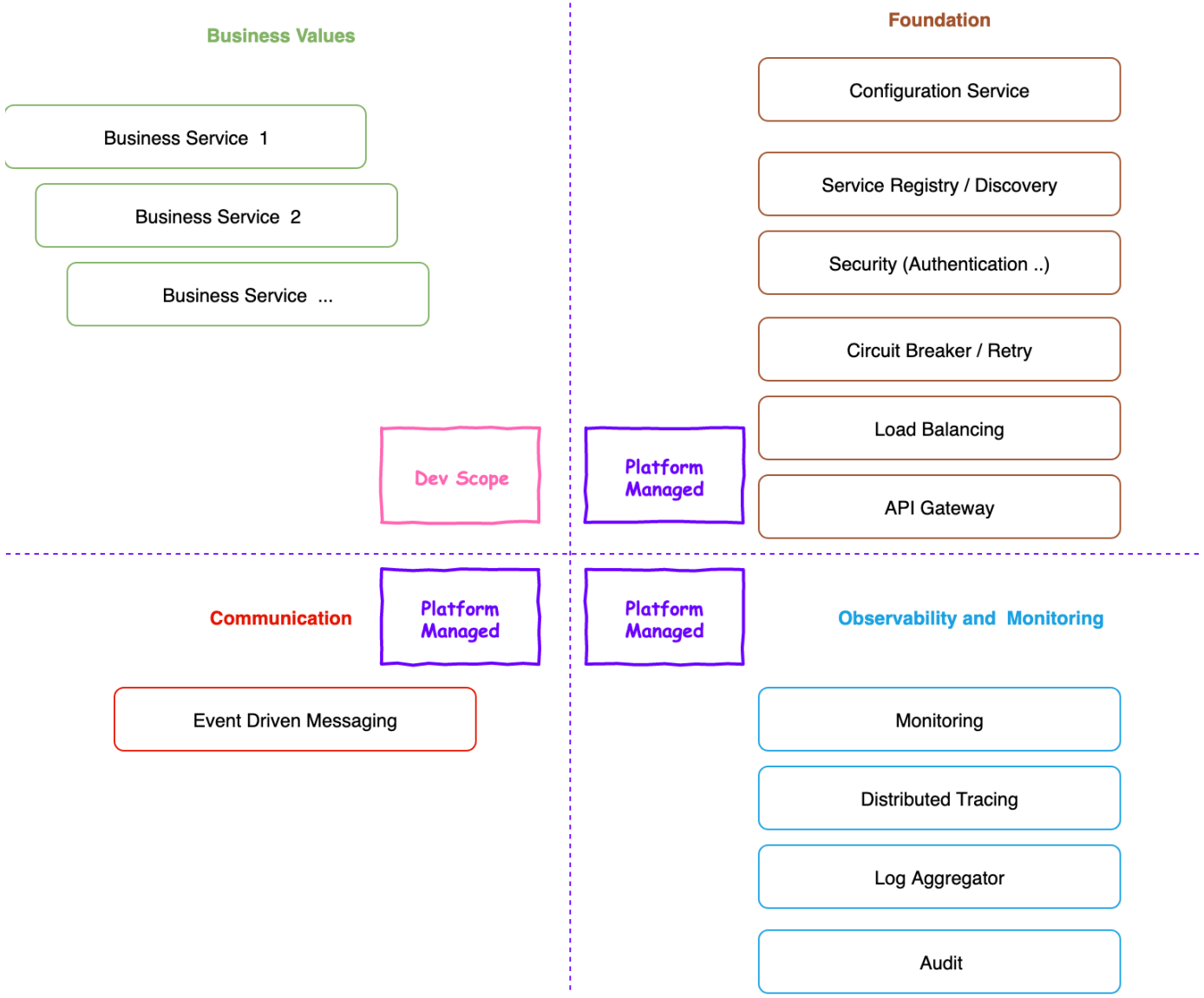
- Delivering business services aligned with capabilities and outcomes
- Designing APIs & contracts for clear integration points
- Owning data & business logic without worrying about platform details
- Driving innovation by iterating quickly and safely

Microservice platform overview

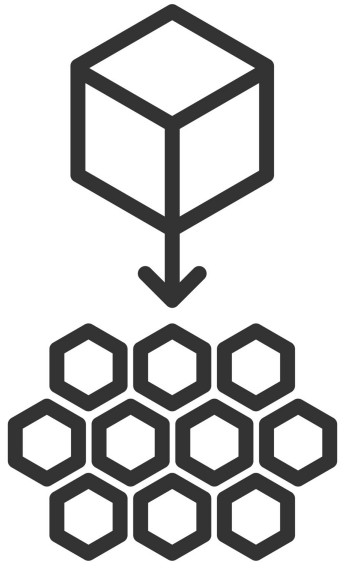
The platform provides the technical foundation, while the services team focuses on delivering the business services that create visible value.

This separation allows developers to concentrate on features while the platform manages the complexity of running distributed systems.

- Business services focus on customer-facing capabilities and outcomes
- The foundation provides configuration, service discovery, security, and routing
- Communication is supported through event-driven messaging between services
- Observability enables monitoring, tracing, logging, and auditing to keep the system healthy



Code oriented pattern

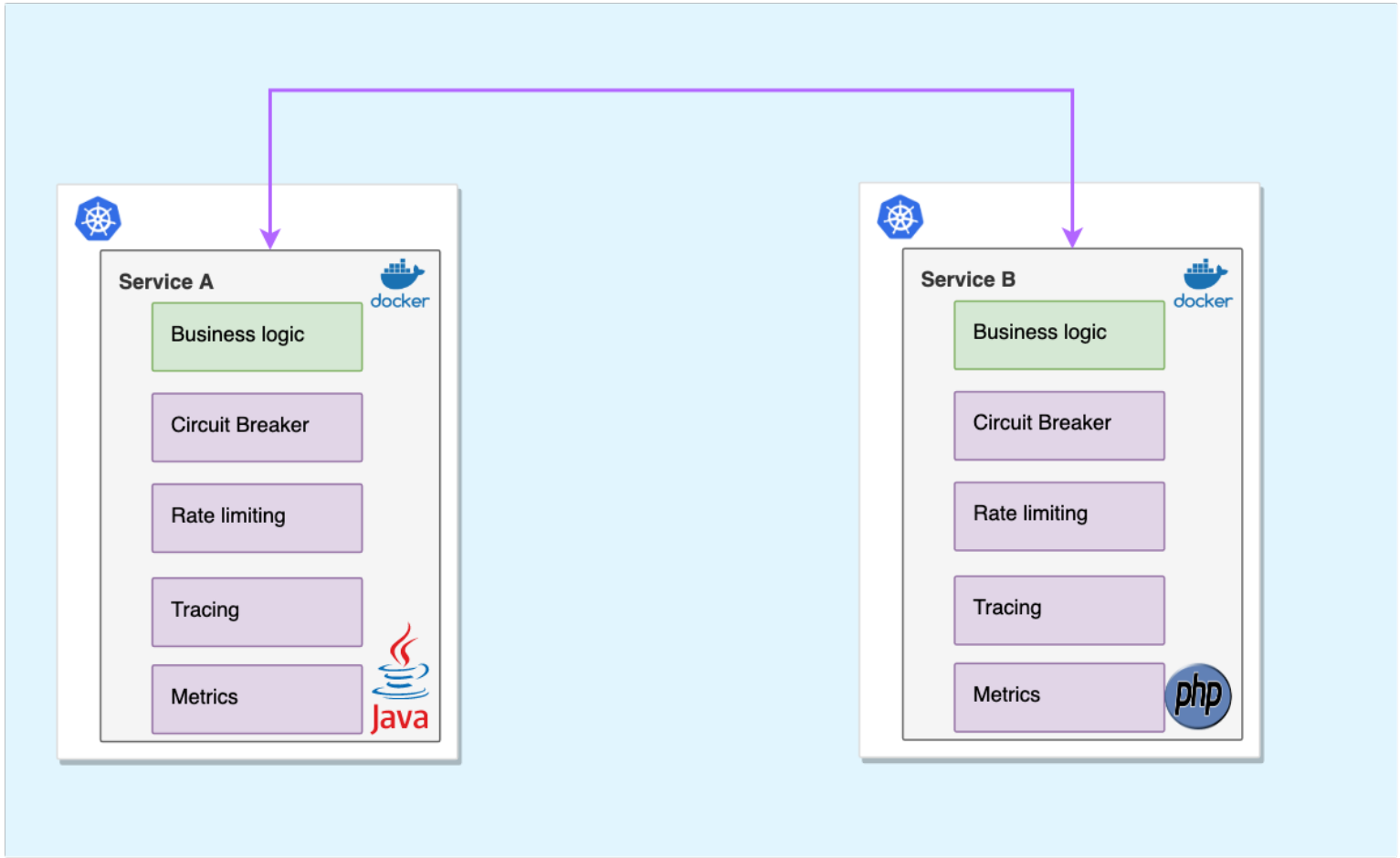


Microservice Architecture

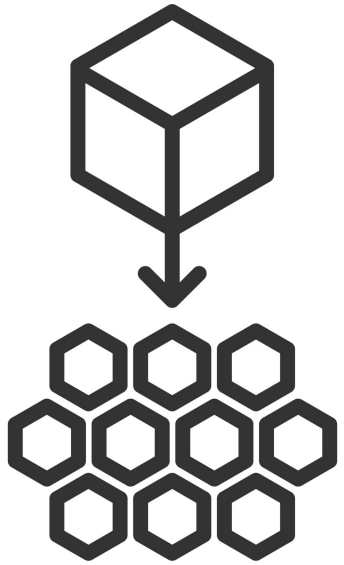
Early efforts to address these concerns focused on building libraries that provided the needed capabilities.
This approach relied on embedding the libraries directly into the service's base code to supply functions such as configuration, communication, and resilience.

Code oriented pattern

Each service includes its own infrastructure libraries alongside business logic. Circuit breakers, rate limiting, tracing, and metrics are implemented inside each codebase, often in different languages and stacks. This approach works but duplicates effort and creates inconsistent implementations across services.



Netflix OSS and Spring Cloud



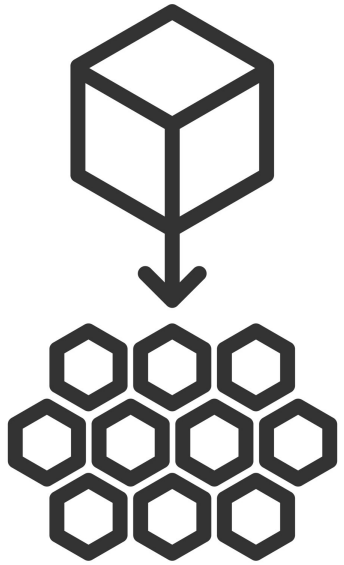
Microservice Architecture

Netflix OSS and later Spring Cloud became the most popular and widely adopted frameworks in the Java ecosystem.

They provided ready-made solutions for common microservice needs such as service discovery, circuit breakers, routing, and configuration.

- Enabled Java teams to build microservices faster
- Simplified integration with other Netflix OSS components
- Became the default choice for many Java-based platforms

Limits of the library-oriented pattern

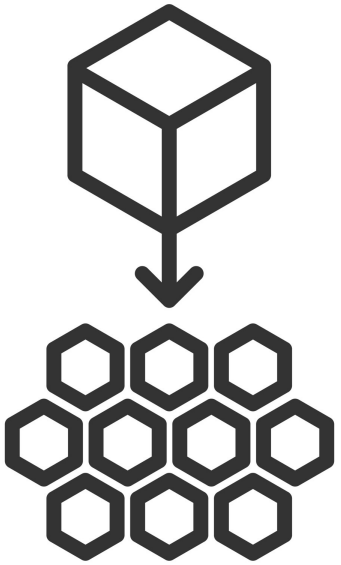


Microservice Architecture

As systems grew, the library-based approach revealed its inherent limitations. Each service embedded its own infrastructure code, leading to increased complexity and inconsistency.

- Strong language dependency — separate libraries were needed for each programming language.
- Prone to errors and inconsistent implementations across teams
- Hard to upgrade libraries across dozens or hundreds of services

Desired state



Microservice Architecture

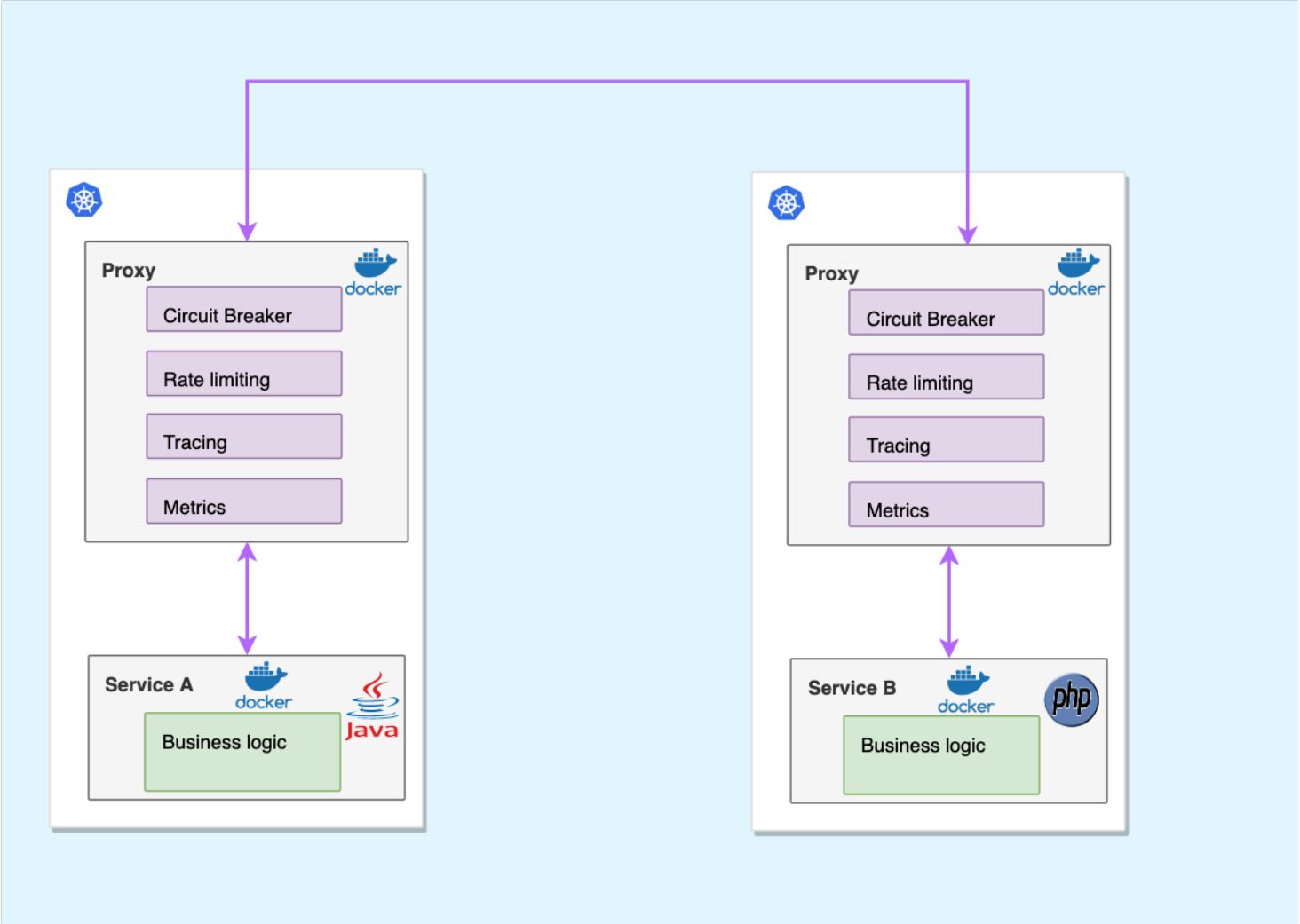
To build scalable and maintainable systems, technical concerns should be separated from business logic, allowing developers to focus on delivering value.

- Keep microservice concerns separate from business logic
- Make the network transparent to applications
- Let developers focus on business capabilities, not infrastructure code
- Ensure service interconnection is language agnostic and easy to upgrade

Sidecar pattern

A lightweight helper container is added to each service to provide common microservice features without changing the service code.

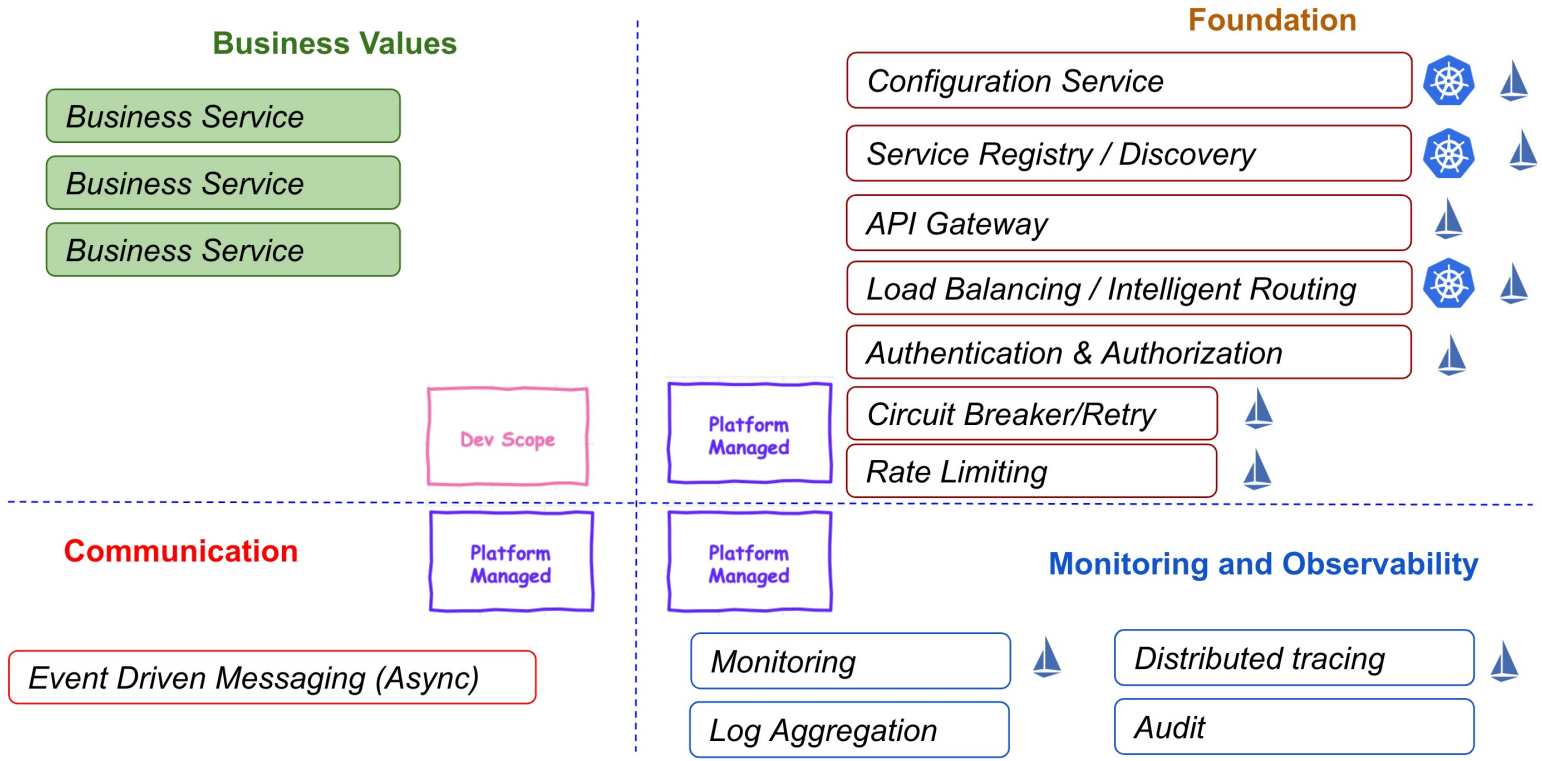
- Delivers circuit-breaking, rate-limiting, tracing, and metrics
- Keeps services focused on business logic
- Works across different languages and frameworks
- Ensures consistent deployment of shared capabilities



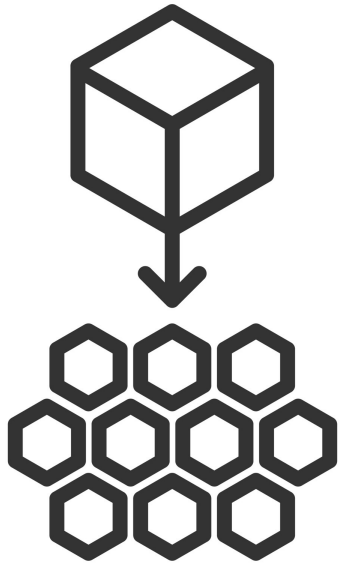
Service mesh benefits

A service mesh is a dedicated infrastructure layer that manages service-to-service communication in modern cloud-native applications, ensuring reliable request delivery across complex service topologies while offloading this responsibility from application code.

- Increases delivery speed and reliability
- Improves security and governance of services
- Simplifies communication management for developers



Istio architecture



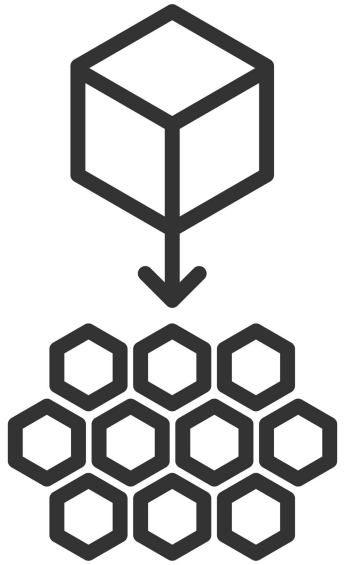
Microservice Architecture

Istio offers a modular architecture similar to Kubernetes, comprising a control plane and a data plane.

This design separates traffic management and policy enforcement from application code, simplifying service-to-service communication.

- The control plane manages and configures the network
- Data plane handles actual service communication
- Sidecar proxies create the service mesh network

Istio control plane

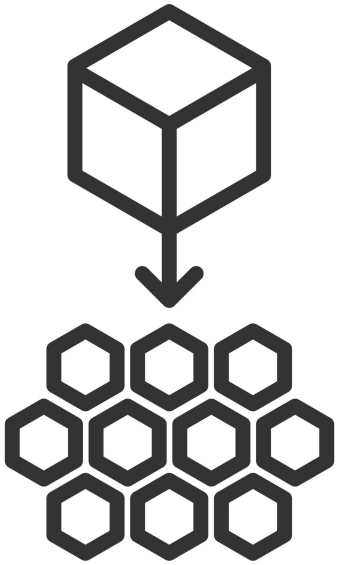


Microservice Architecture

The control plane is provided primarily by istiod, which manages the mesh and distributes configuration to proxies.

- Service discovery and traffic rules via istiod (xDS to Envoy)
- mTLS and identity via istiod certificates and SDS
- Policy and telemetry configured through Istio APIs and providers

Istio data plane



Microservice Architecture

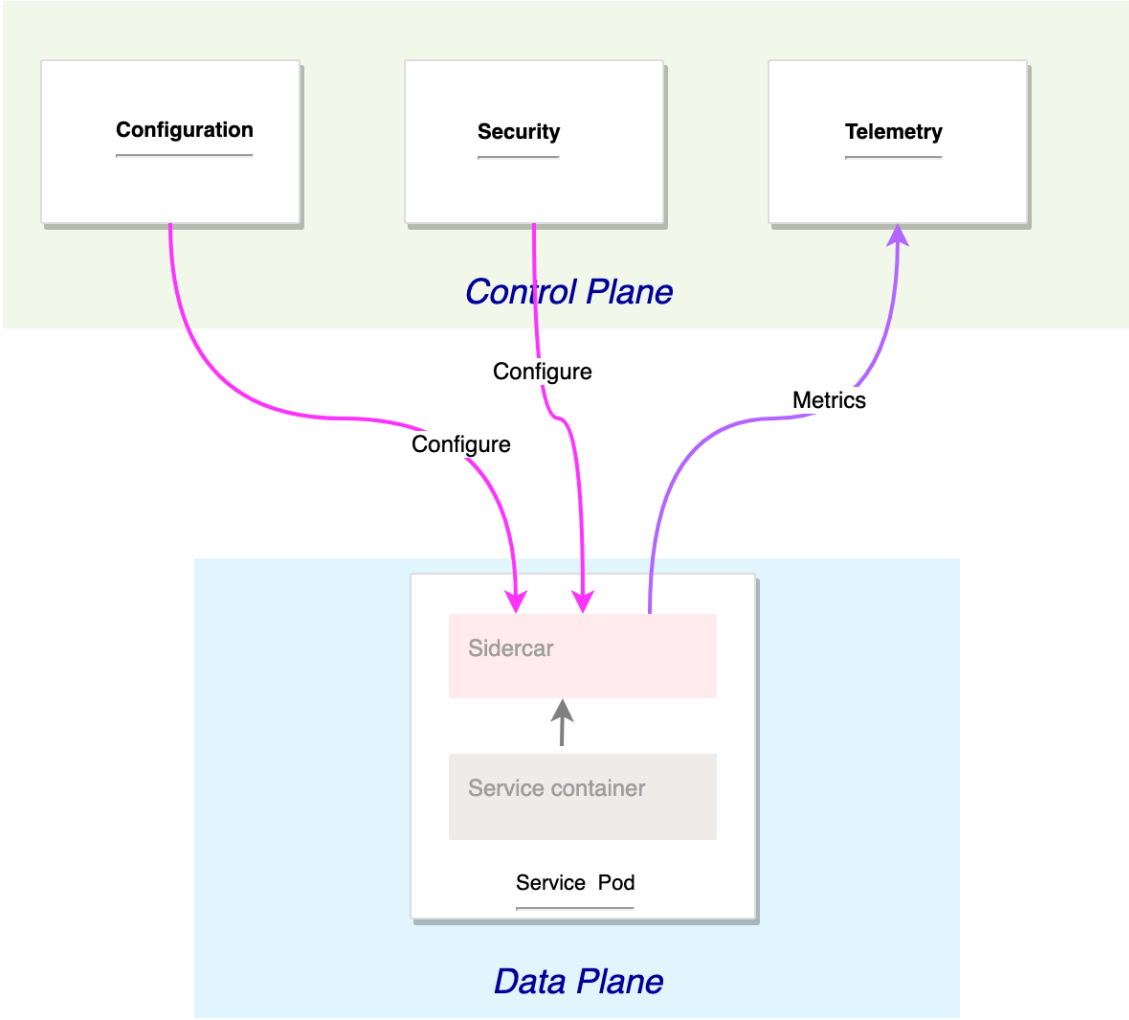
The data plane comprises intelligent Envoy proxies deployed as sidecars alongside each service. These proxies manage all network traffic between microservices and collect operational data.

- Mediates and controls service-to-service communication
- Collects metrics, logs, and traces
- Forms the interconnected service mesh across all services
- Keeps networking logic out of service code

Istio components

Istio uses a control plane to configure and secure sidecar proxies while collecting telemetry, and a data plane where each service pod runs a sidecar alongside the service container.

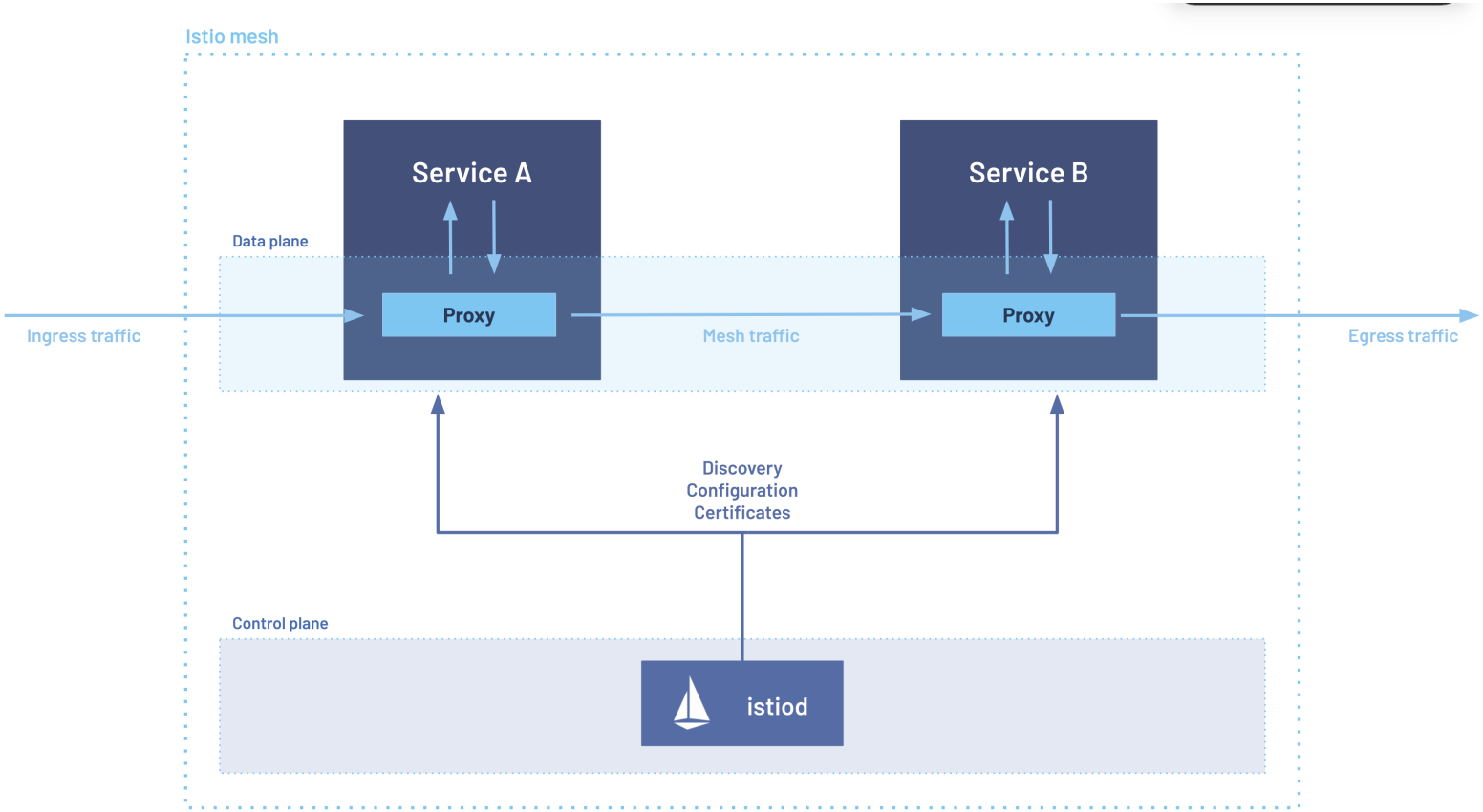
- Configuration and security modules set traffic rules and enforce policies
- The telemetry module gathers metrics and reports them to the control plane
- Sidecar proxies handle all service-to-service communication inside the mesh



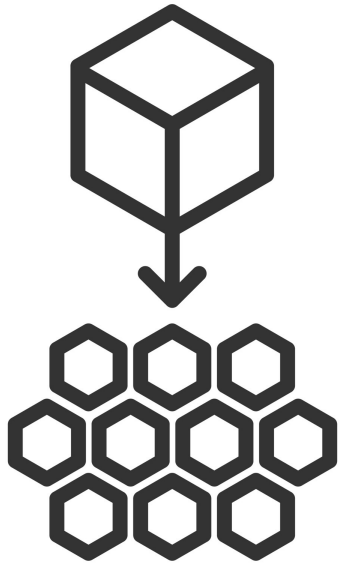
Istio components

Istio uses a control plane to configure and secure sidecar proxies while collecting telemetry, and a data plane where each service pod runs a sidecar alongside the service container.

- Configuration and security modules set traffic rules and enforce policies
- The telemetry module gathers metrics and reports them to the control plane
- Sidecar proxies handle all service-to-service communication inside the mesh



Service mesh overview

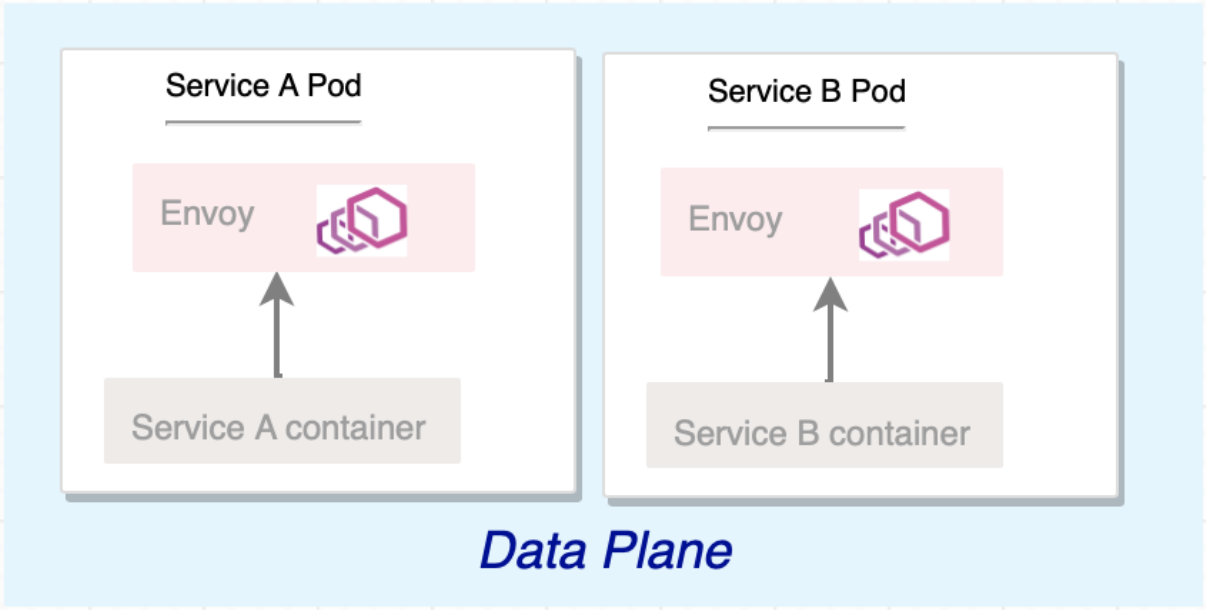


Microservice Architecture

A service mesh manages communication between microservices using two main layers: the control plane and the data plane.
The control plane provides configuration, security, and telemetry, while the data plane uses sidecar proxies to handle traffic between services.

- Separates service networking from application code
- Centralizes traffic control, security, and observability
- Supports reliable communication across complex service topologies

Service mesh overview



Dashboards

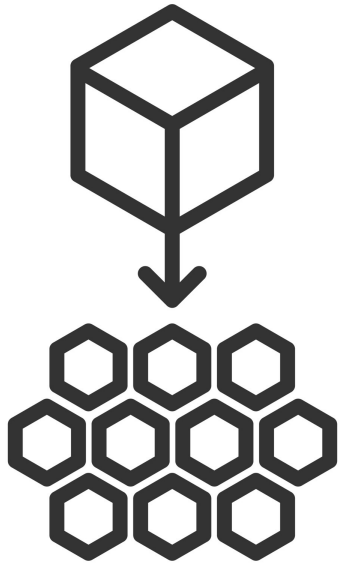
Monitoring

Tracing

Observability

Service Graph

Istio control plane components

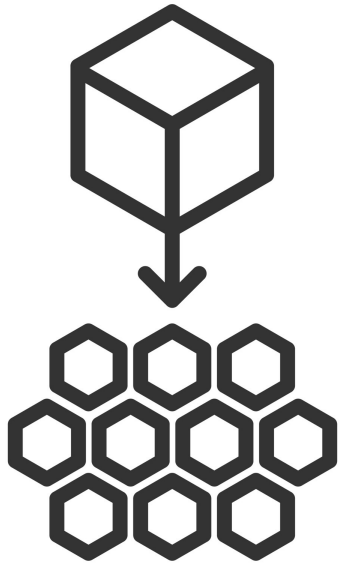


Microservice Architecture

Istio uses a control plane to configure and secure sidecar proxies while collecting telemetry, and a data plane where each service pod runs an Envoy sidecar alongside the service container.

- Configuration and security modules set traffic rules and enforce policies
- Telemetry gathers metrics and reports to the control plane
- Sidecar proxies handle service-to-service communication

Istio data plane components

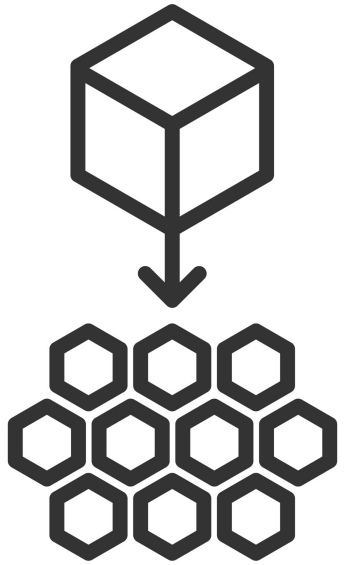


Microservice Architecture

The data plane consists of service pods, each running an Envoy sidecar that handles all inbound and outbound traffic.
Dashboards give visibility into mesh health and traffic patterns.

- Envoy manages inbound and outbound traffic for each service
- Policies and traffic rules are applied at the proxy
- Telemetry is collected from real traffic

Istio control plane traffic flow

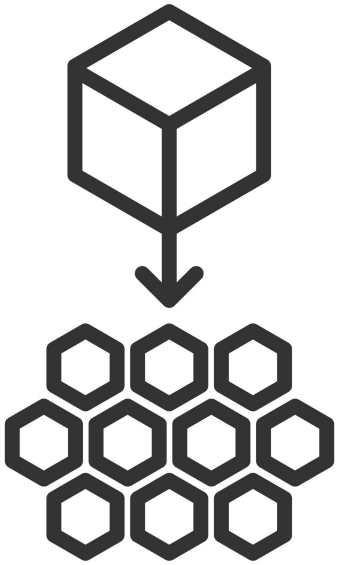


Microservice Architecture

The control plane defines how traffic enters and exits the mesh, as well as how configuration is distributed to all proxies.

- Ingress Gateway allows and routes inbound traffic to mesh services using Gateway and VirtualService
- Egress Gateway and ServiceEntry control outbound access to external systems such as databases and third-party APIs
- istiod pulls from the Kubernetes API server and pushes config to Envoy sidecars across the mesh

Sidecar proxies



Microservice Architecture

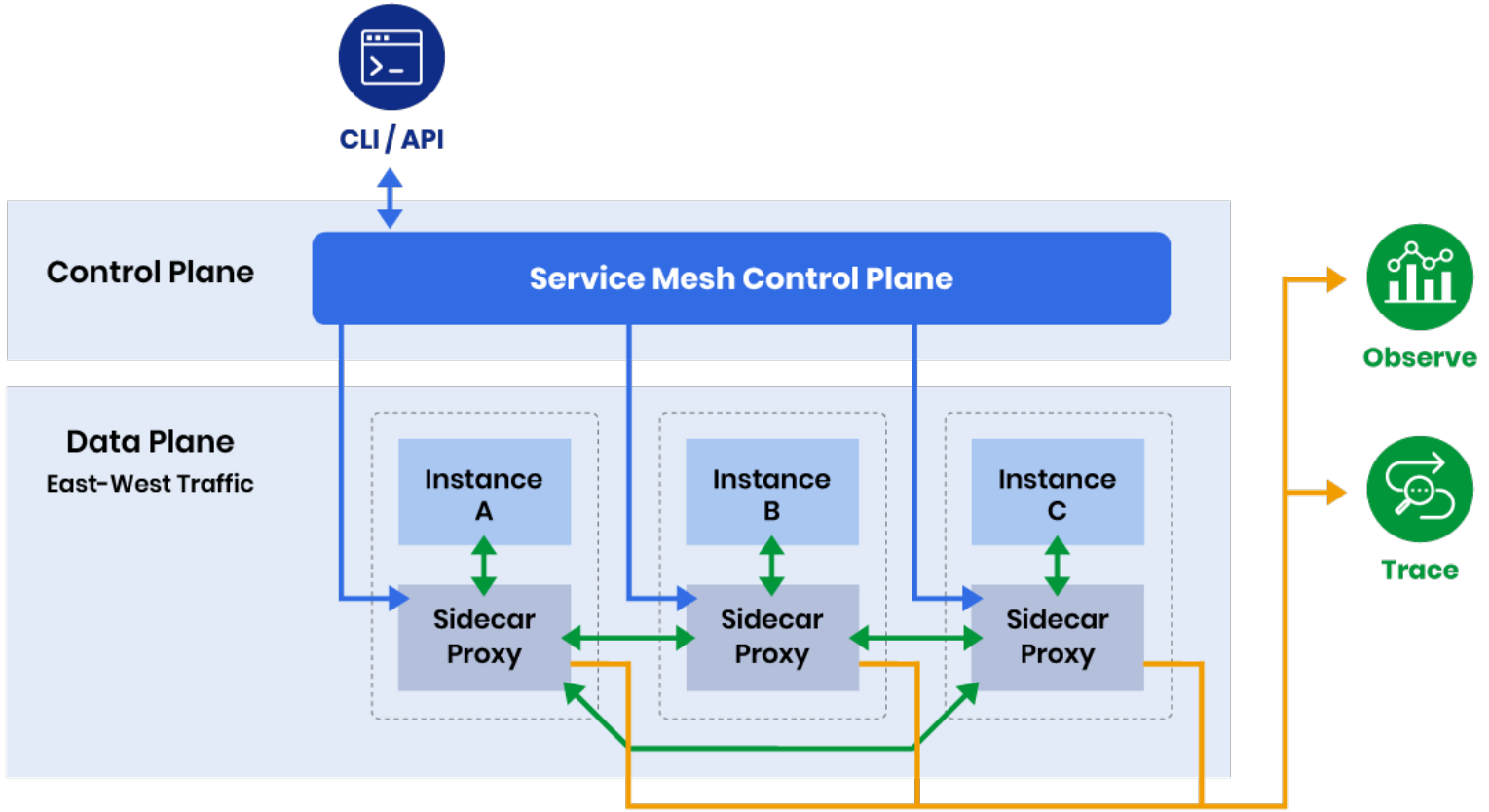
Every microservice pod includes an Envoy sidecar that receives configuration from the control plane, allowing the service to focus on its business logic.

- Simplifies service communication
- Adds resilience with retries and circuit breaking
- Removes networking logic from application code

Istio architecture

The control plane manages traffic rules and security, while the data plane uses sidecar proxies to handle service communication and collect metrics.

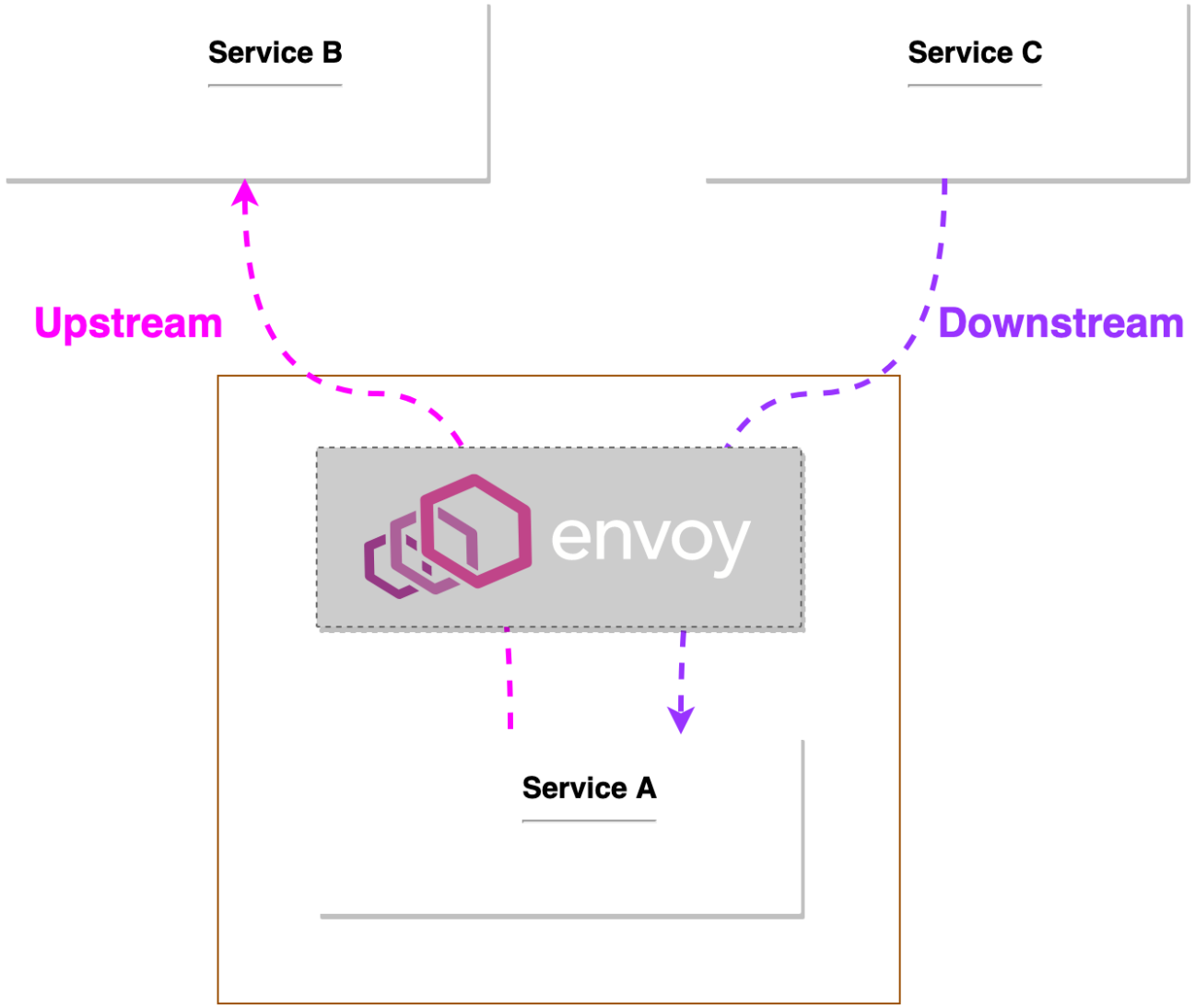
- Control plane configures and secures service-to-service calls
- Data plane routes east–west traffic through Envoy sidecars
- Metrics and traces provide visibility and debugging insights



Upstream vs downstream

Envoy classifies every network connection as upstream or downstream, helping define how requests move through the service mesh and how routing, policy, and telemetry are applied.

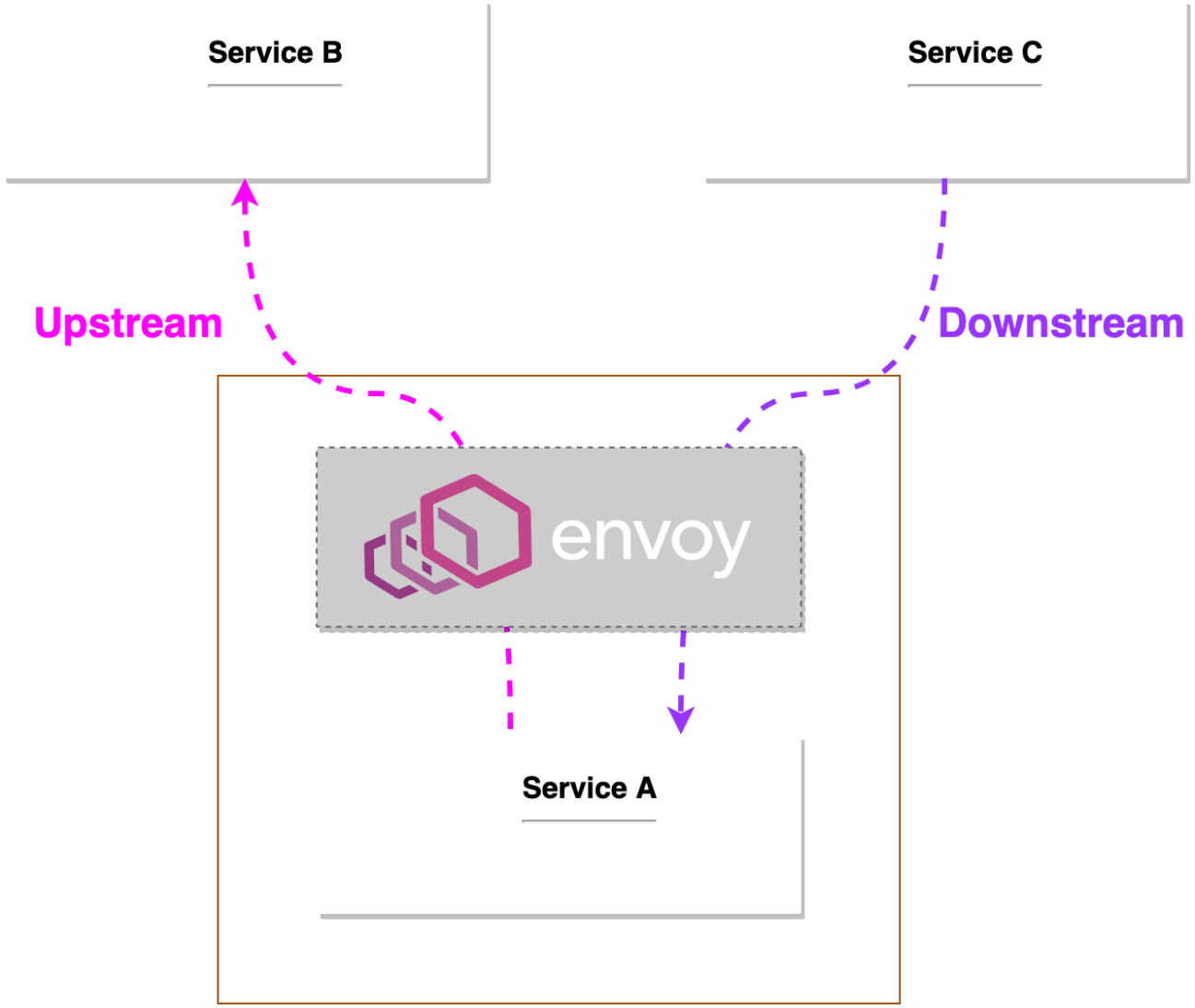
- Upstream is the service Envoy connects to
- Downstream is the client sending a request to Envoy
- This model drives routing, policy, and telemetry decisions



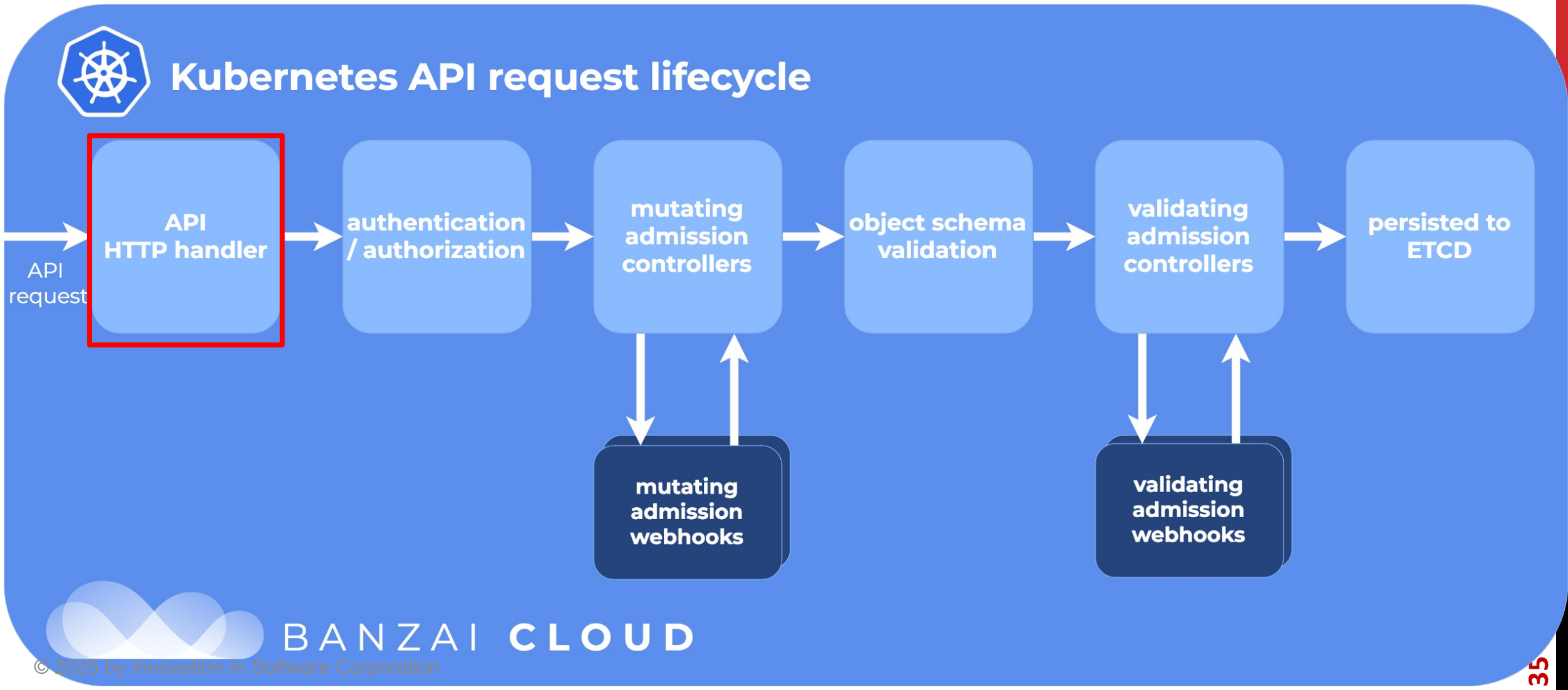
Envoy traffic flow

Each microservice runs an Envoy sidecar that manages all inbound and outbound traffic, applying routing rules, enforcing security, and collecting observability data for the mesh.

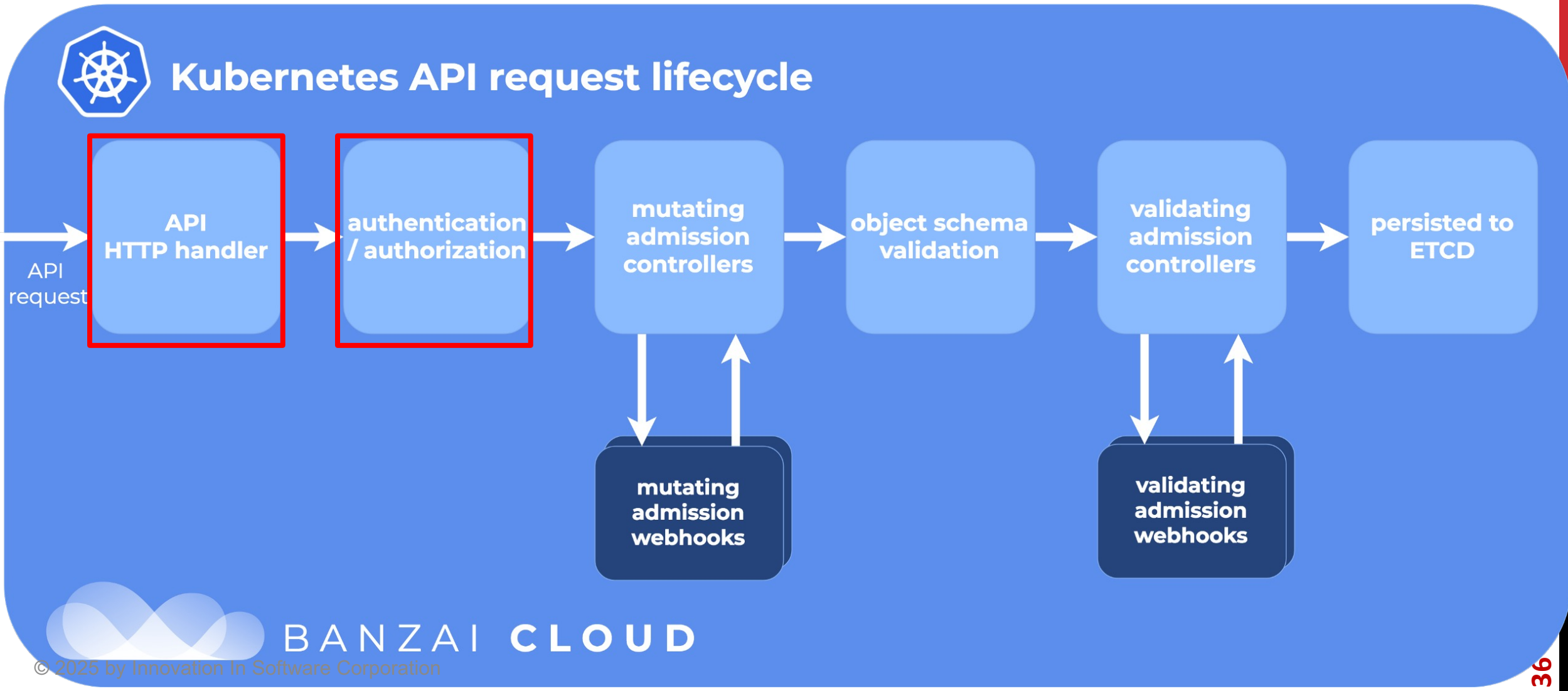
- Downstream traffic arrives from the client into the proxy
- Upstream traffic leaves the proxy toward another service
- Envoy applies routing rules, security, and observability on every request



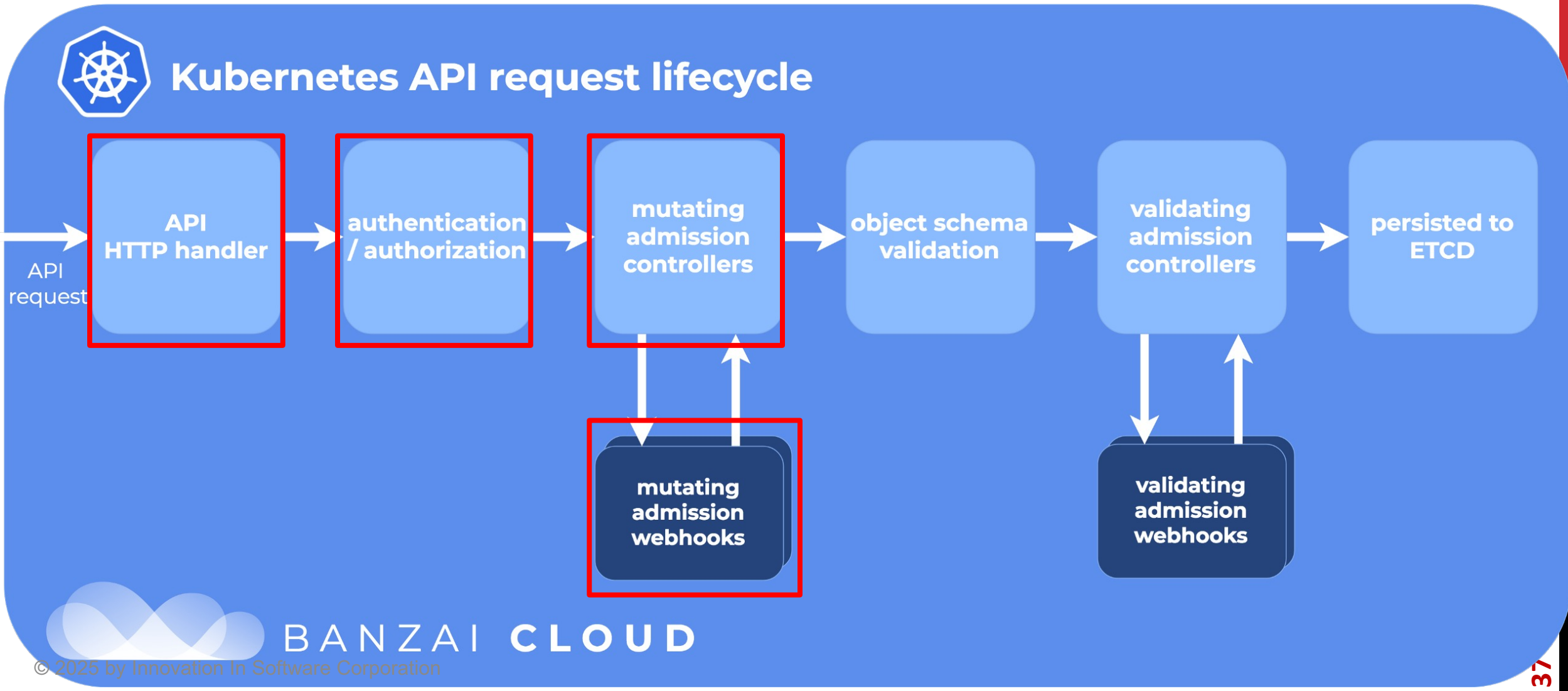
Kubernetes lifecycle



Kubernetes lifecycle

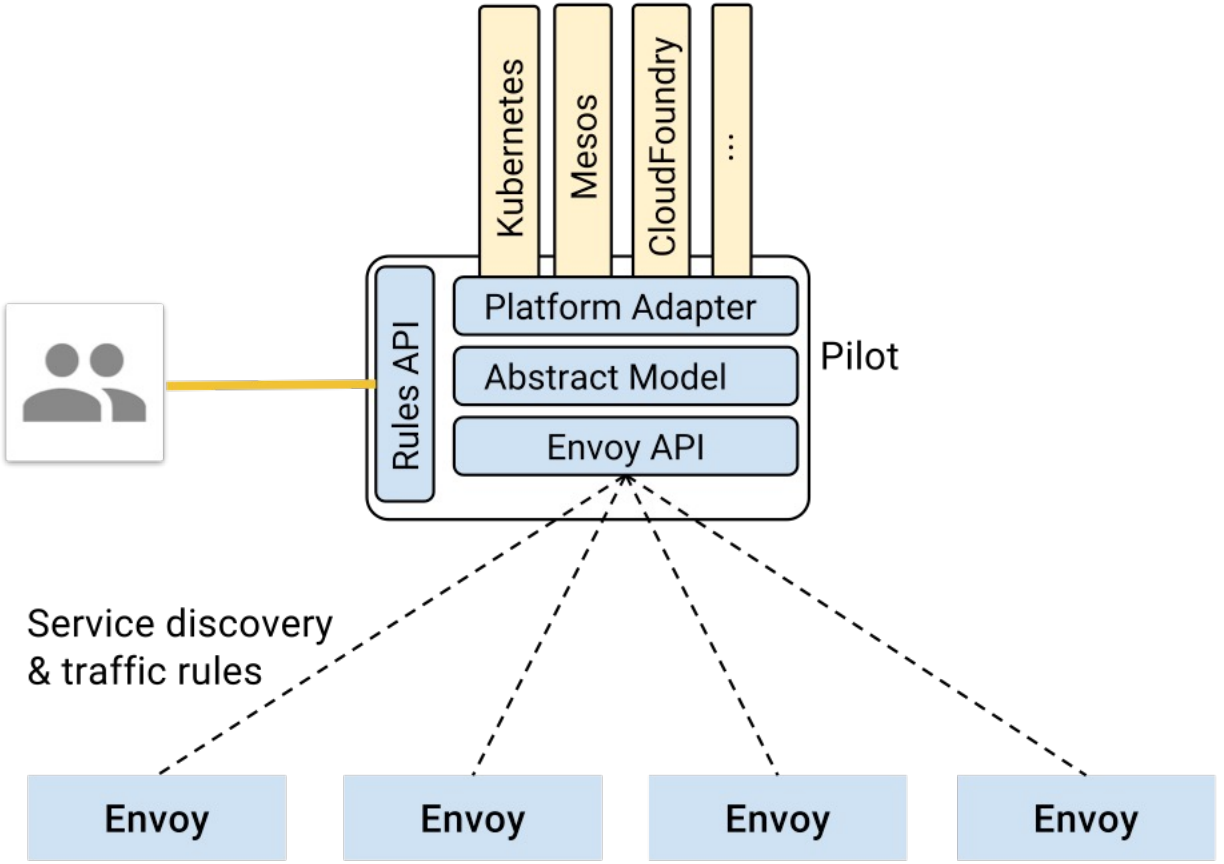


Kubernetes lifecycle



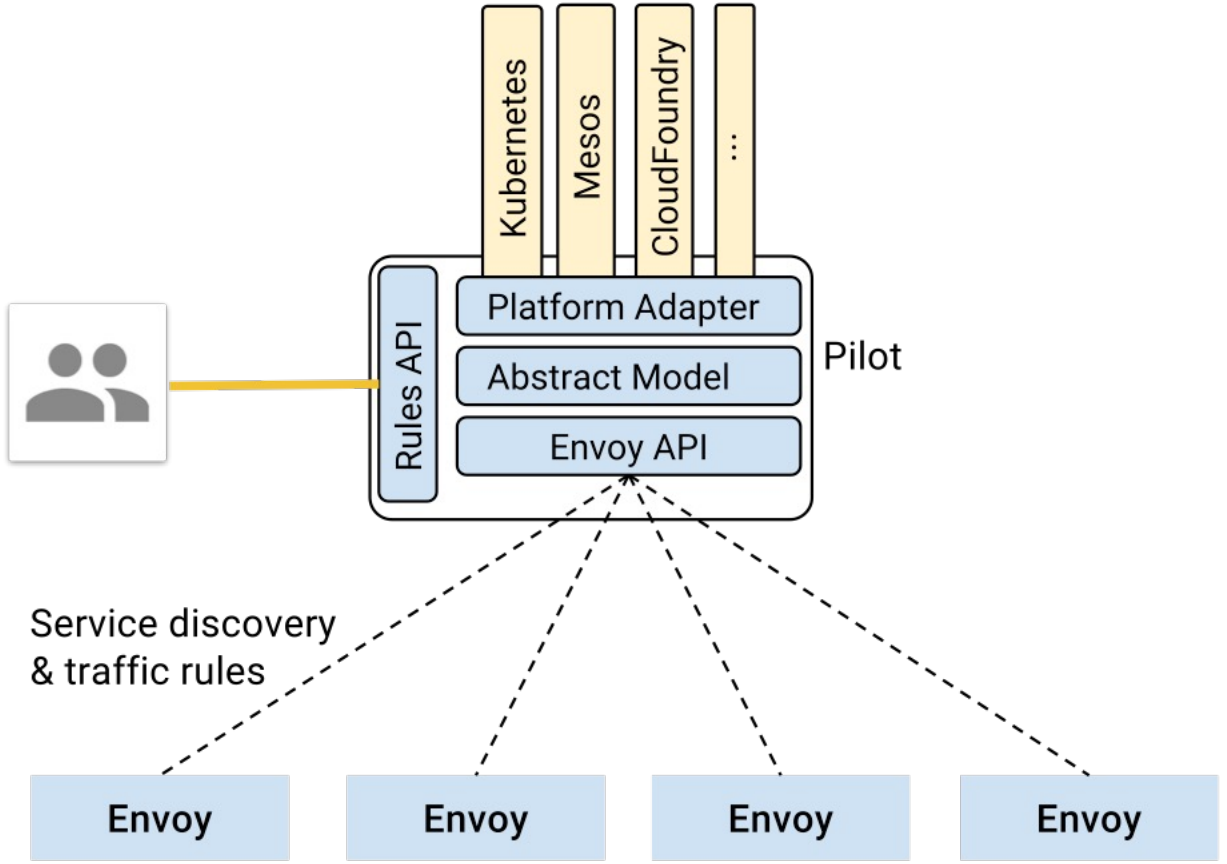
Istiod - Pilot

- Manages Envoy proxies
- Exposes APIs
  - Service discovery
  - dynamic updates (LB pools, routing tables)
- Independent of the underlying platform
- Each platform has its own adapters



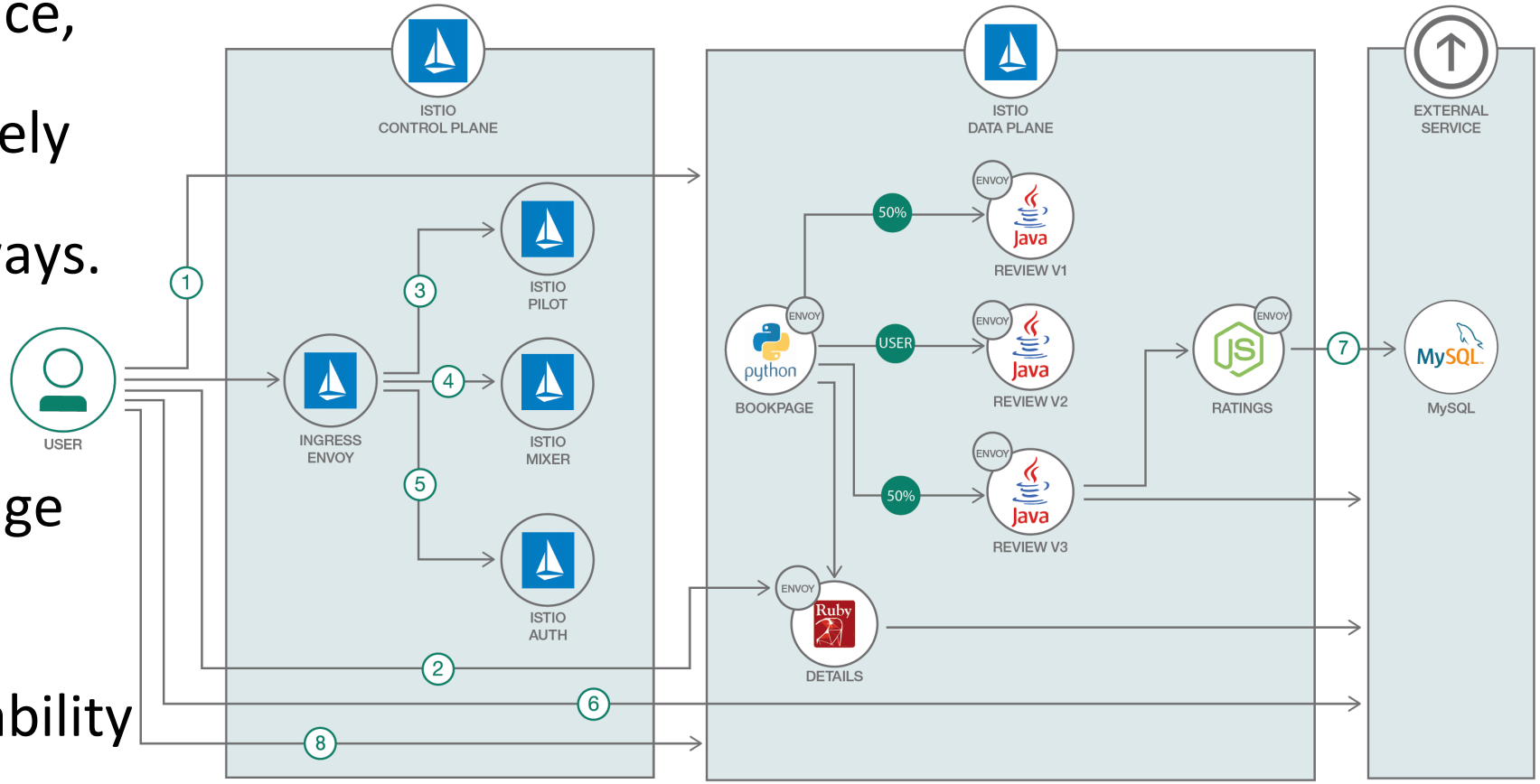
Istiod - Pilot

- Kubernetes adapter watches Kubernetes API for changes to:
  - Pod
  - Service registration
  - ingress
  - traffic management rules



Envoy

Envoy is a high-performance, CNCF-graduated proxy originally built at Lyft, widely used as the data plane for service meshes and gateways.



- Runs as a sidecar or edge gateway
- Provides L7 routing, resilience, and observability

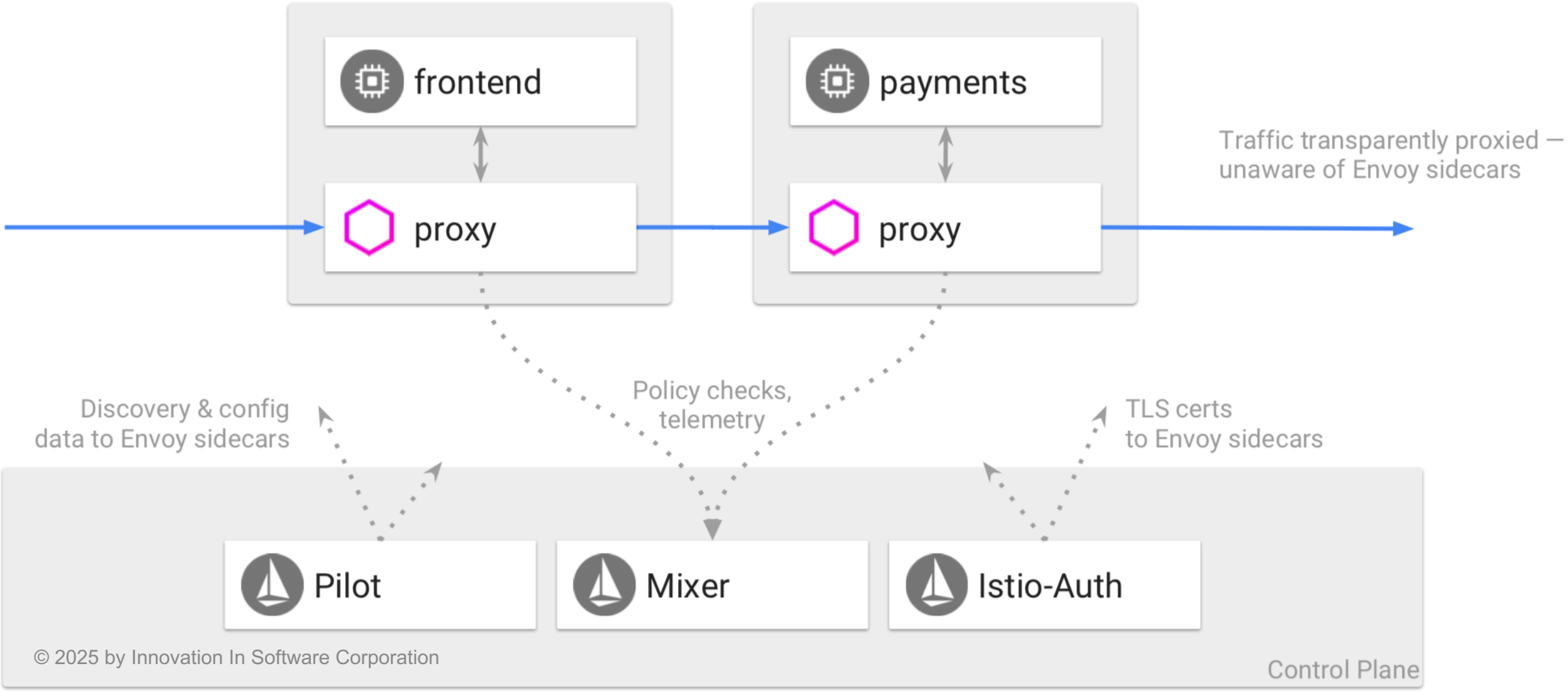
Envoy

```
spec:
  containers:
  - image: frontend:latest
```

```
spec:
  containers:
  - image: frontend:latest
  - image: istio/proxy
```


```
initImage: docker.io/istio/proxy_init
  proxyImage: docker.io/istio/proxy
```

Envoy



Lab: Install Istio and deploy application

Traffic Management - Rules

Control how API calls & layer-4 traffic flows between services in application deployment

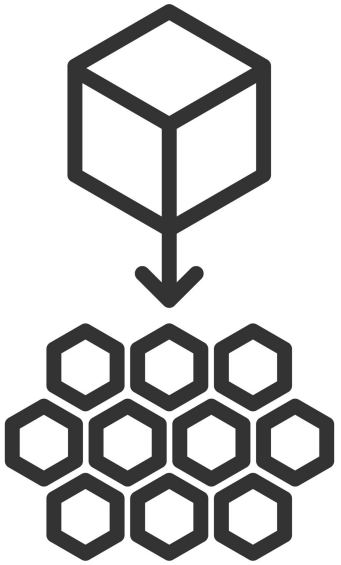
- Set service-level properties
  - timeouts
  - retries
  - circuit breakers
- Continuous deployment
  - canary
  - A/B
  - staged rollouts (% based traffic splits)

Traffic Management - VirtualService

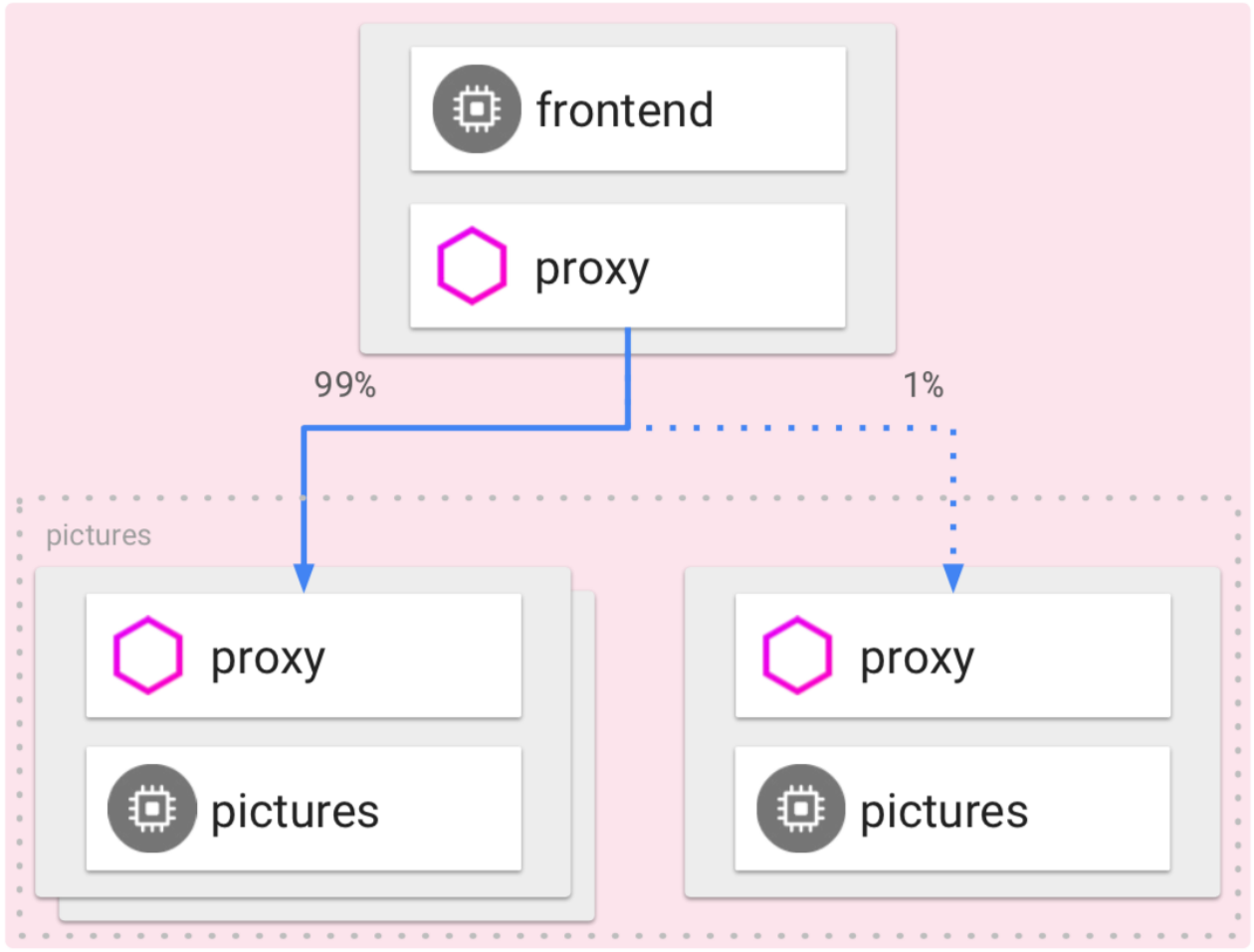
Defines rules that control how requests are routed within an Istio service mesh

- source
- destination
- HTTP paths
- header fields

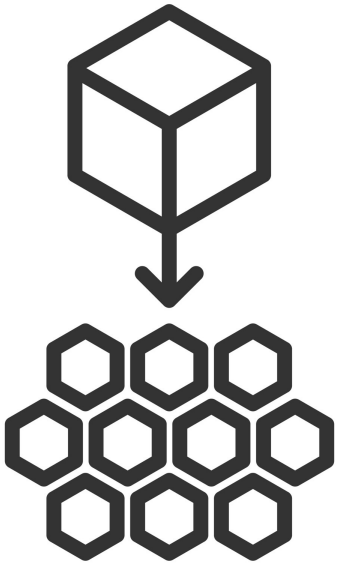
VirtualService



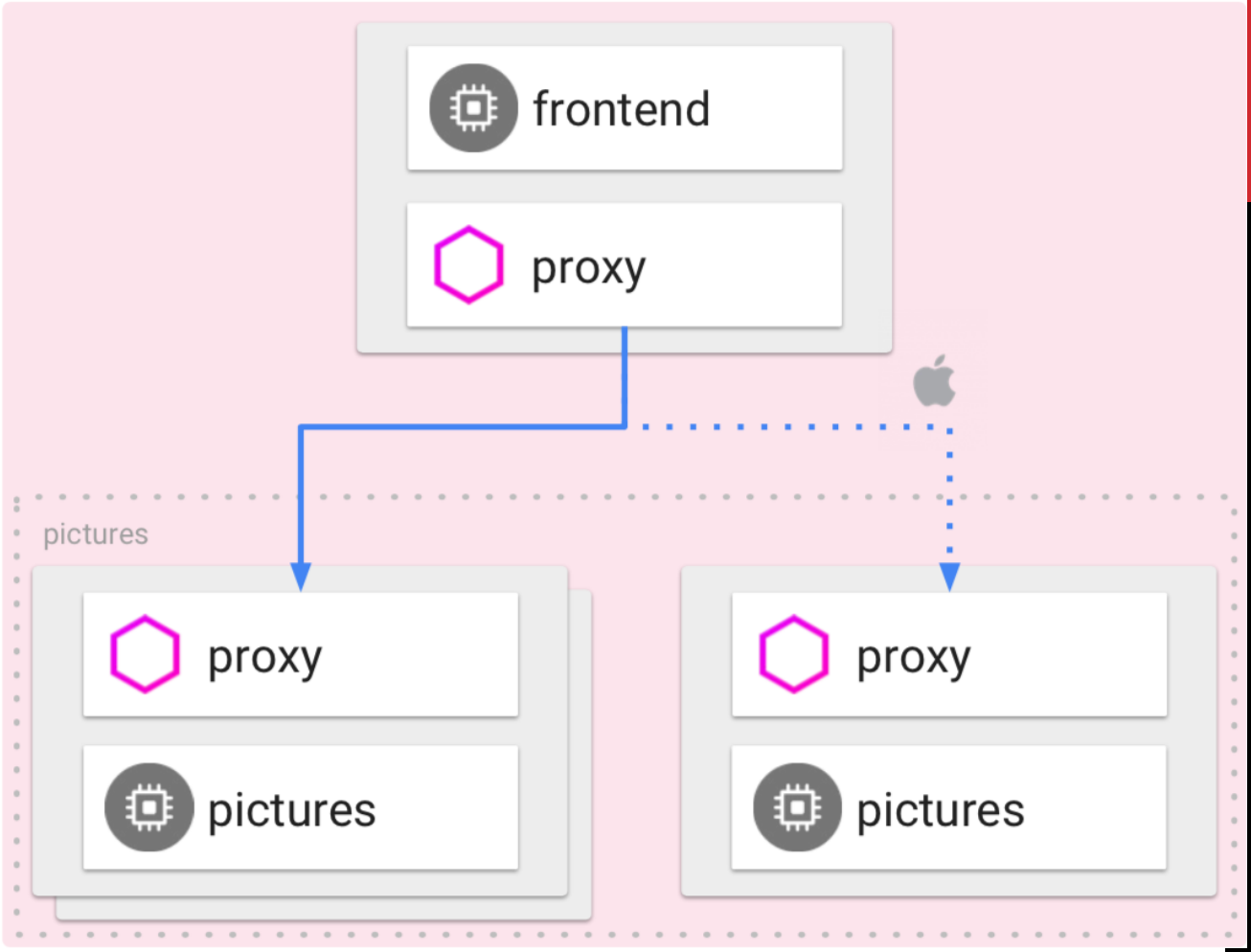
Microservice Architecture



VirtualService



Microservice Architecture



VirtualService- config

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews
  namespace: default
spec:
  hosts:
  - reviews.default.svc.cluster.local  # FQDN recommended
  http:
  - route:
    - destination:
        host: reviews.default.svc.cluster.local
        subset: v1
```

VirtualService- config

The route subset specifies the name of a defined subset in a corresponding destination rule configuration:

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews
  namespace: default
spec:
  hosts:
  - reviews.default.svc.cluster.local  # FQDN recommended
  http:
  - route:
    - destination:
        host: reviews.default.svc.cluster.local
      subset: v1
```

Traffic Management - DestinationRule

Correspond to one or more request destination hosts that are specified in a VirtualService configuration

- Hosts may or may not match actual destination workload
- Route to services in mesh or external targets
- Prefer FQDNs; short names work in the same namespace but can be ambiguous

```
hosts:
  - reviews
  - bookinfo.com
```

Traffic Management - DestinationRule

This DestinationRule tells Istio how to route traffic for the reviews service.
It defines two subsets, v1 and v2, based on the version label on the pods.

VirtualServices can use these subsets to control traffic flow, such as splitting requests between v1 and v2.

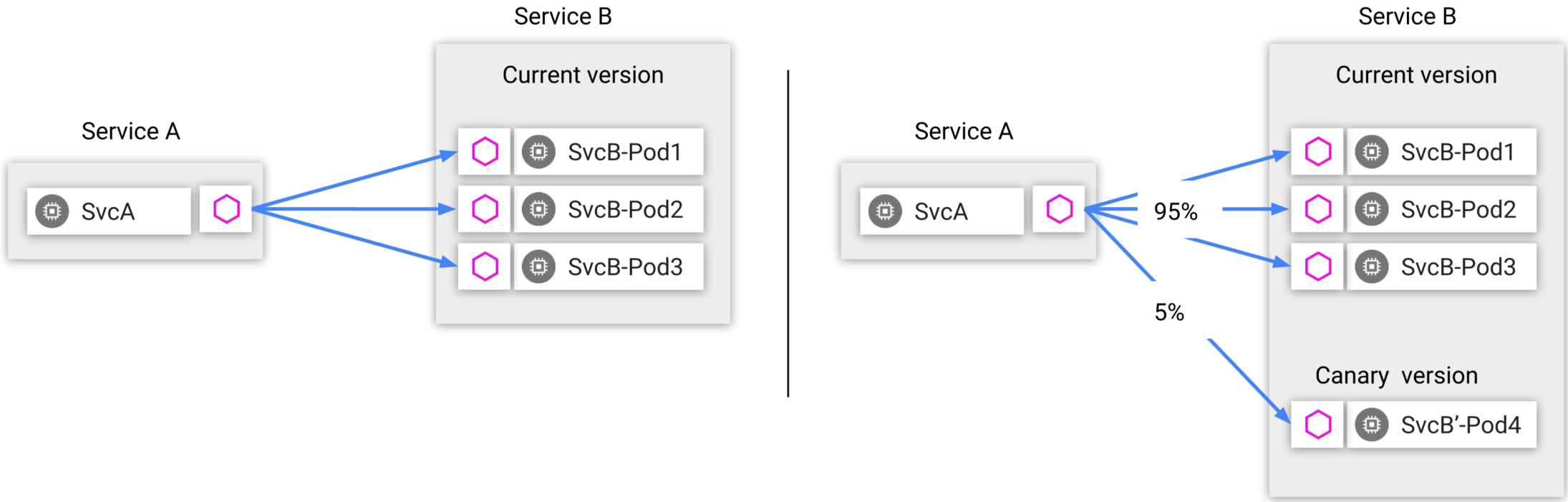
```
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: reviews
spec:
  host: reviews.default.svc.cluster.local
  subsets:
    - name: v1
      labels:
        version: v1
    - name: v2
      labels:
        version: v2
```

Traffic Management - config

```
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: reviews
  namespace: default
spec:
  host: reviews.default.svc.cluster.local
  trafficPolicy:
    loadBalancer:
      simple: ROUND_ROBIN
  subsets:
  - name: v1
    labels: { version: v1 }
  - name: v2
    labels: { version: v2 }
  - name: mobile
    labels: { version: mobile }
```

Corresponding DestinationRule with matching subset 'v1'

Traffic Management



Traffic splitting decoupled from infrastructure scaling - proportion of traffic routed to a version is independent of number of instances supporting the version

Qualify Rules - Specific caller

```yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: ratings
spec:
  hosts:
  - ratings
  http:
  - match:
      sourceLabels:
        app: reviews
    ...
```

Qualify Rules - Specific caller

```yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: ratings
spec:
  hosts:
  - ratings
  http:
  - match:
    - sourceLabels:
        app: reviews
    ...
```

- sourceLabels: matches Kubernetes service selector labels

Qualify Rules - HTTP headers

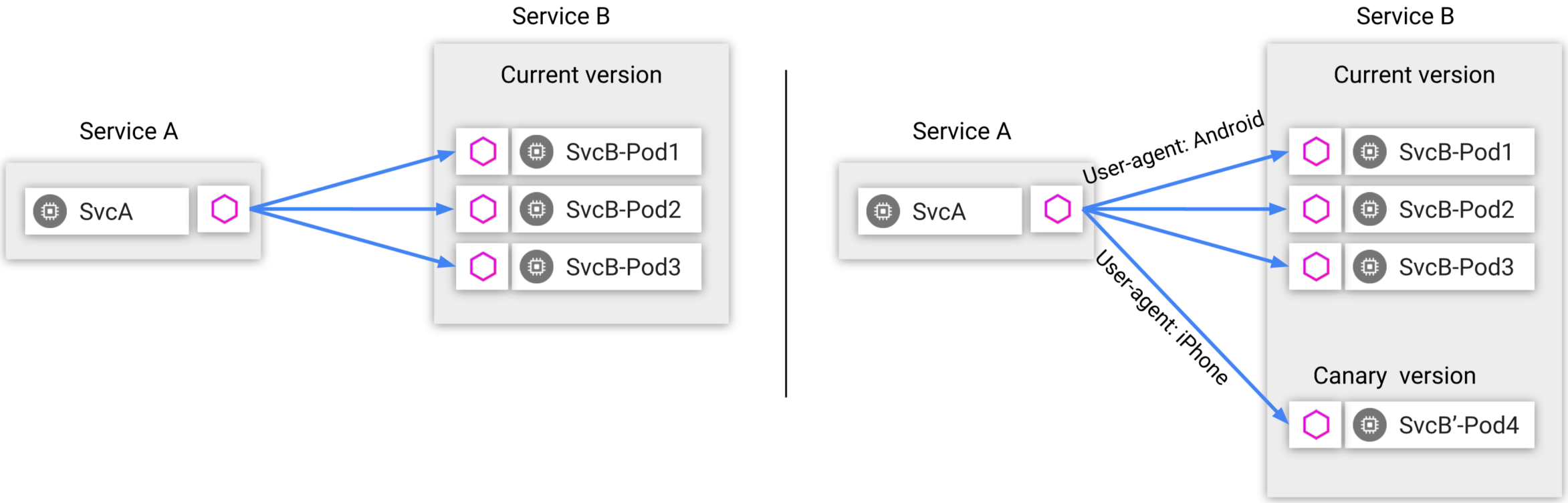
```yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
    - reviews
  http:
  - match:
    - headers:
        cookie:
          regex: "^(.*?;)?(user=jason)(;.*)?$"
    ...
```

Qualify Rules - HTTP headers

```yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
    - reviews
  http:
  - match:
    - headers:
        cookie:
          regex: "^(.*?;)?(user=jason)(;.*)?$"
    ...
```

- Match user=jason HTTP header

Qualify Rules - HTTP headers



Content-based traffic steering - The content of a request can be used to determine the destination of a request

Qualify Rules - HTTP headers

```yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: ratings
spec:
  hosts:
  - ratings
  http:
  - match:
    - sourceLabels:
        app: reviews
        version: v2
      headers:
        cookie:
          regex: "^(.*?;)?(user=jason)(;.*)?$"
    ...
```

Qualify Rules - HTTP headers

```yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: ratings
spec:
  hosts:
  - ratings
  http:
  - match:
    - sourceLabels:
        app: reviews
        version: v2
      headers:
        cookie:
          regex: "^(.*?;)?(user=jason)(;.*)?$"
    ...
```

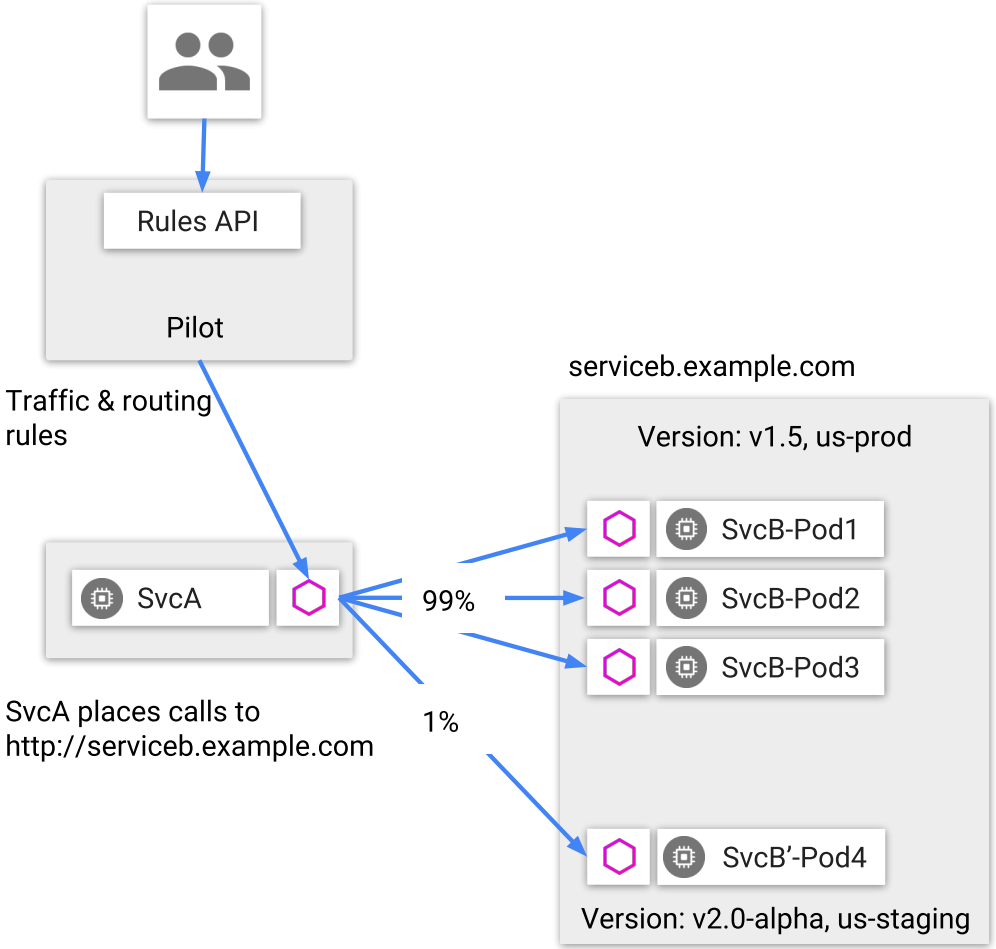
- Multiple conditions in same 'match' (AND)

Qualify Rules - HTTP headers

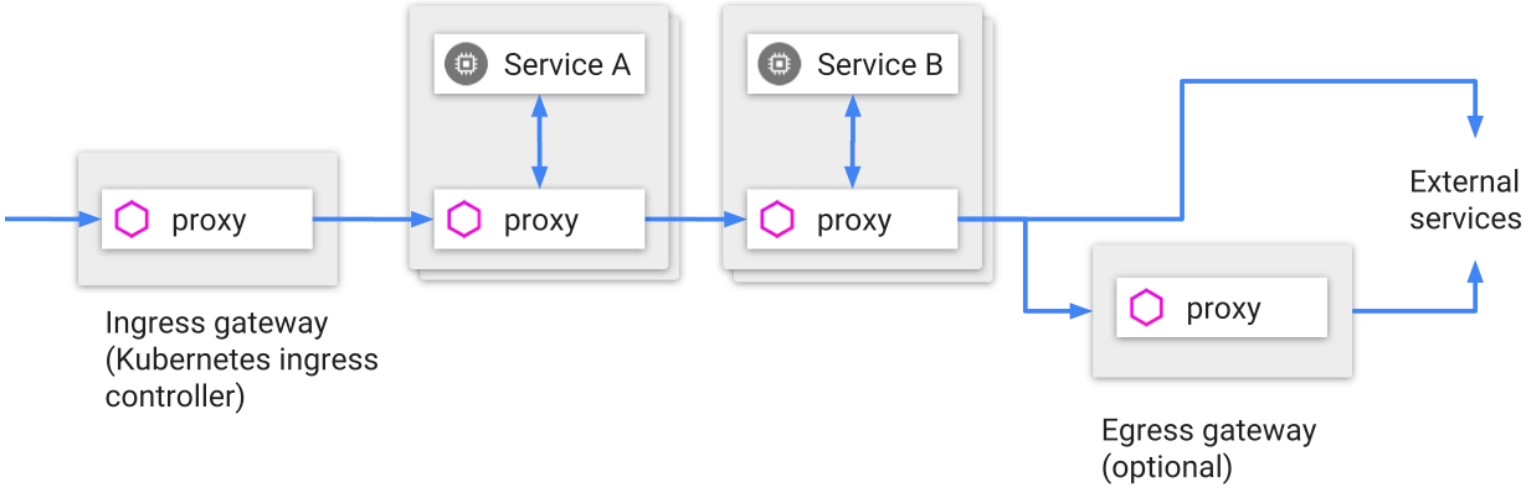
```yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: ratings
spec:
  hosts:
  - ratings
  http:
  - match:
    - sourceLabels:
        app: reviews
        version: v2
  - match:
    - headers:
        cookie:
          regex: "^(.*?;)?(user=jason)(;.*)?$"
    ...
```

- Multiple 'match' (OR)

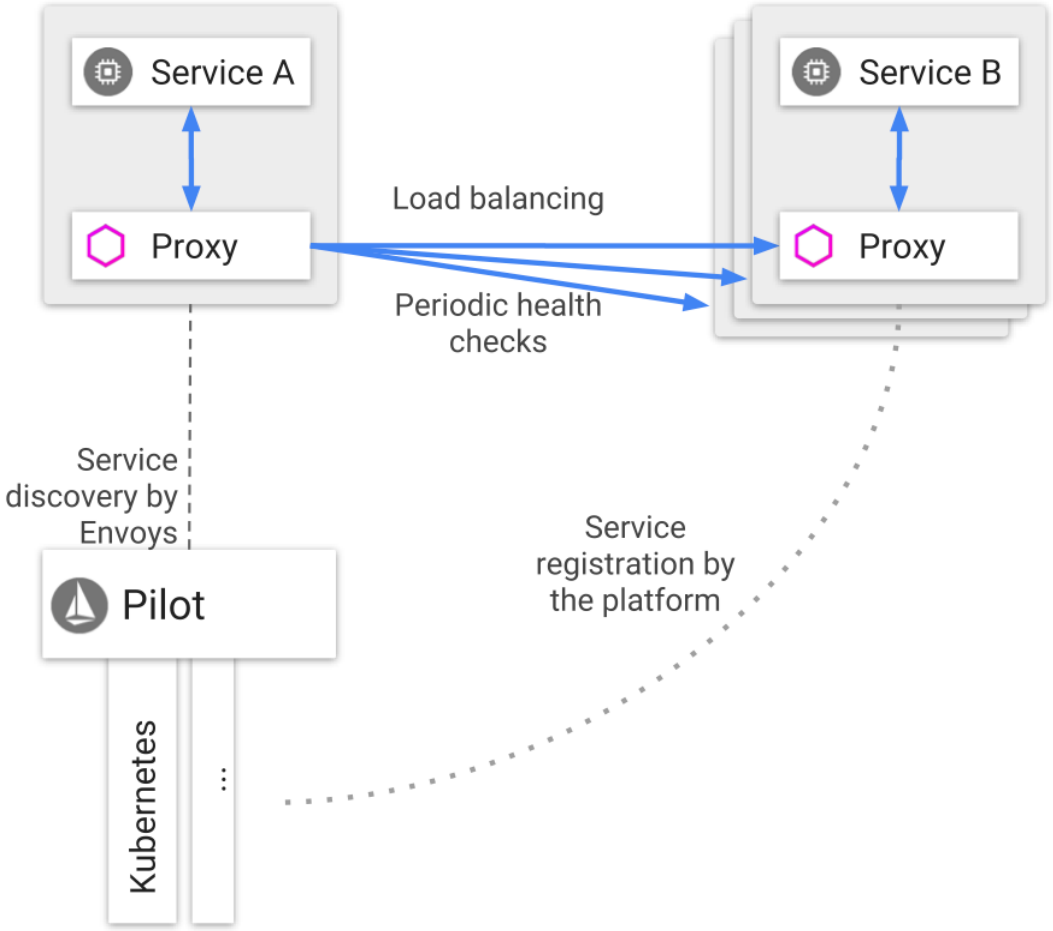
Communication between services



Ingress and egress

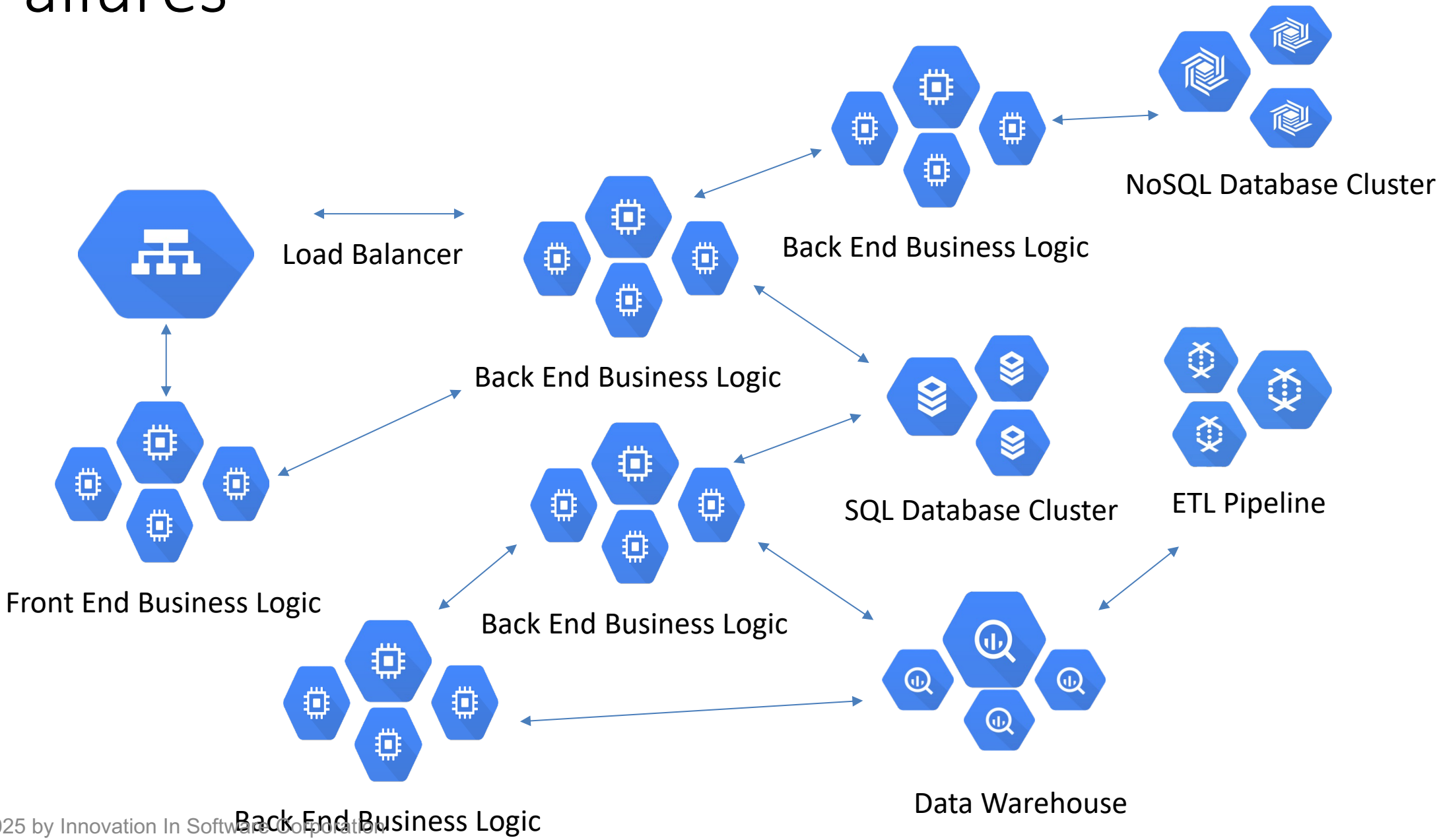


Load balancing

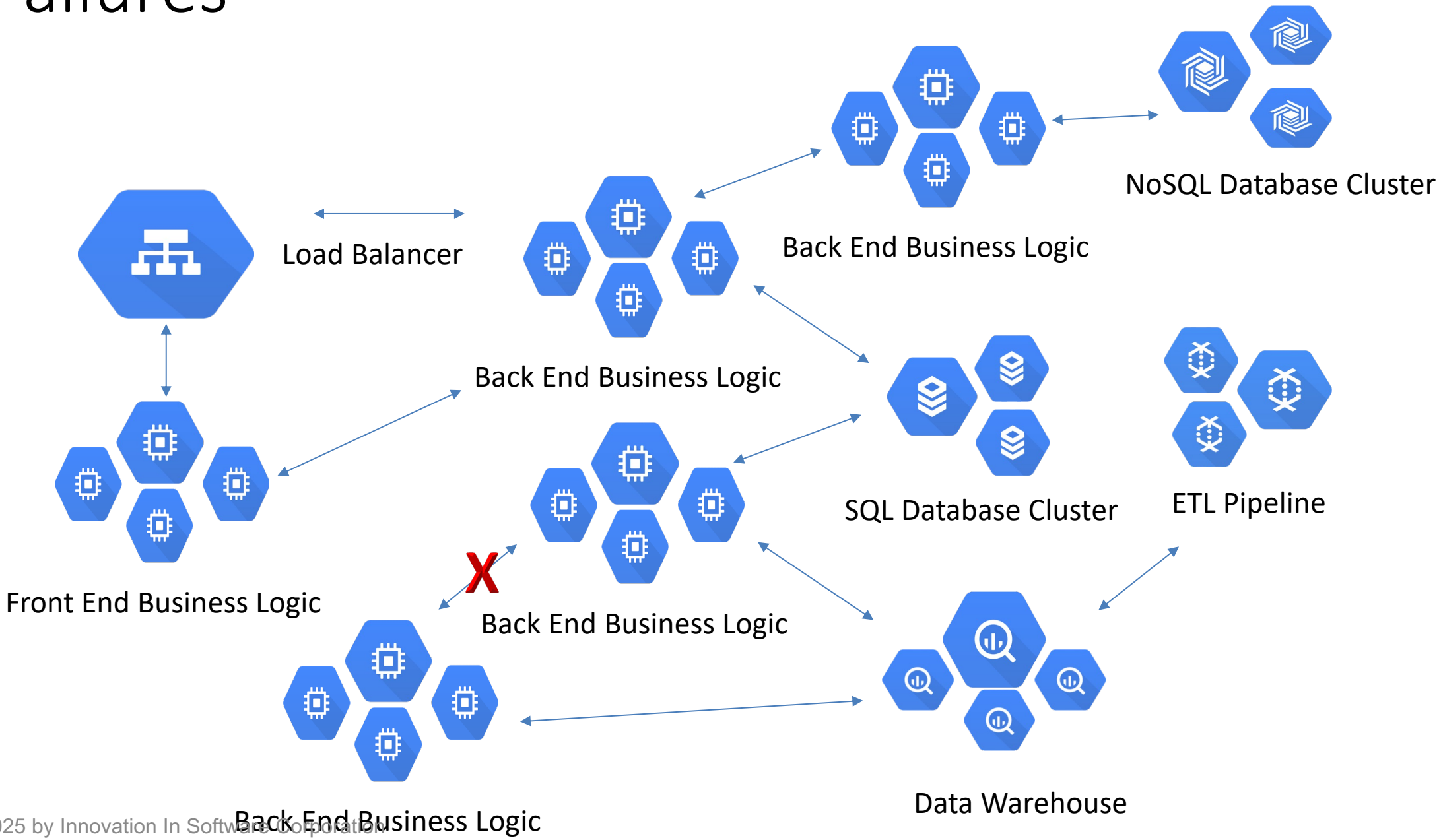


Lab: Traffic management

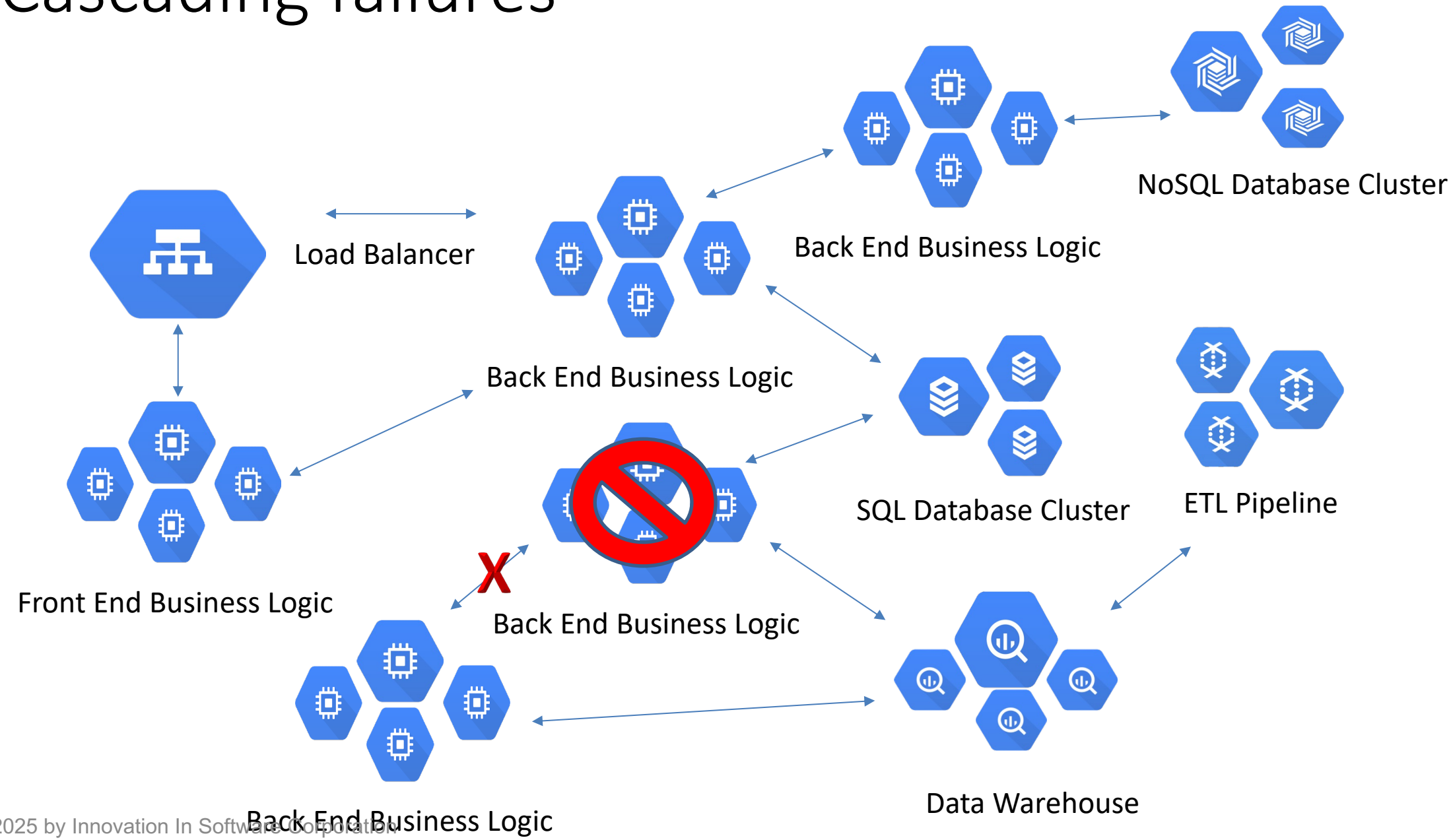
Failures



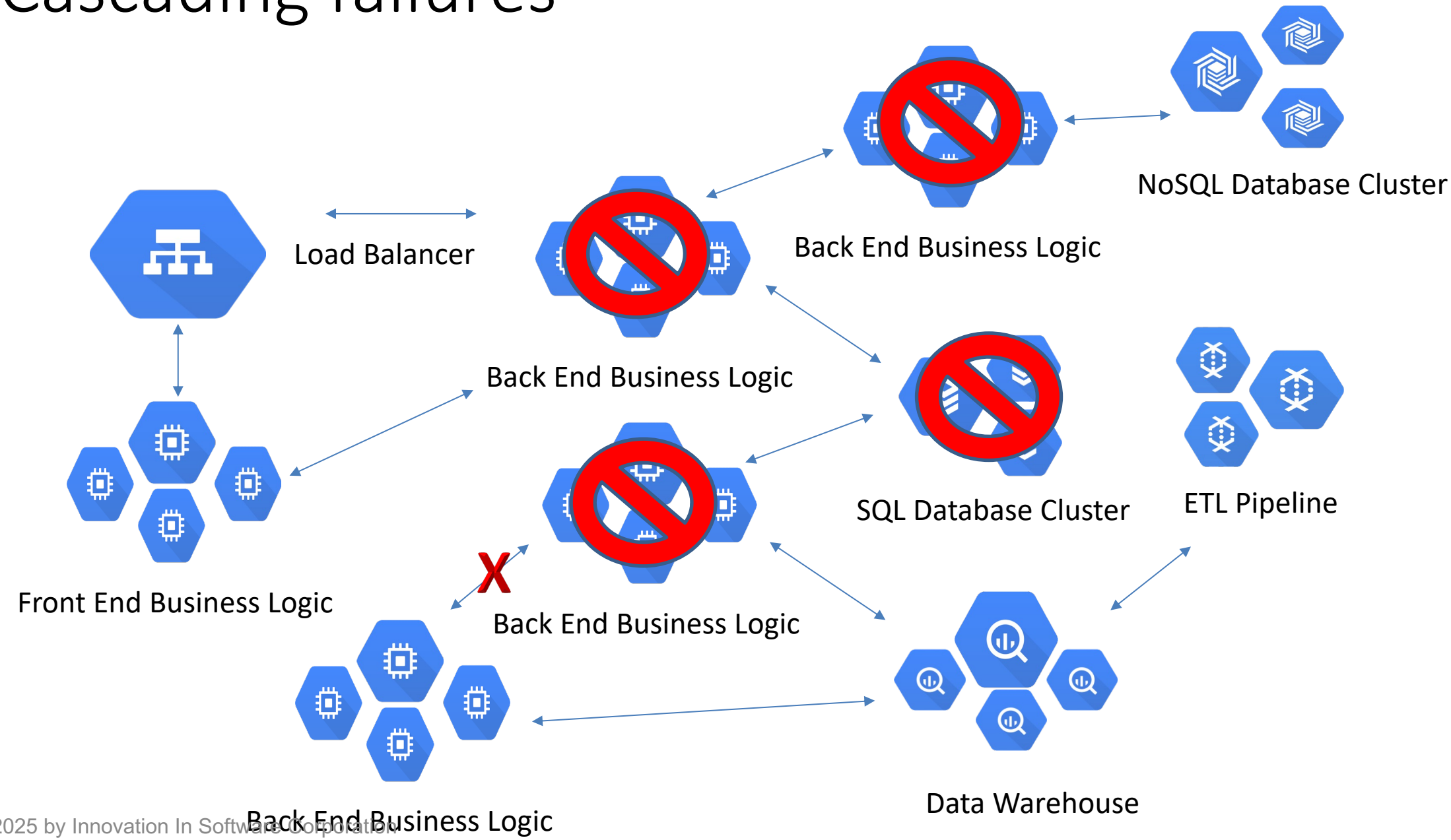
Failures



Cascading failures



Cascading failures



Timeouts & Retries

```yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: ratings
spec:
  hosts:
    - ratings
  http:
  - route:
    - destination:
        host: ratings
        subset: v1
    timeout: 10s
```

Timeouts & Retries

```yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: ratings
spec:
  hosts:
    - ratings
  http:
  - route:
    - destination:
        host: ratings
        subset: v1
    timeout: 10s
```

- Change timeout from default 15 to 10 seconds

Timeouts & Retries

```yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: ratings
spec:
  hosts:
    - ratings
  http:
  - route:
    - destination:
        host: ratings
        subset: v1
    retries:
      attempts: 3
      perTryTimeout: 2s
```

Timeouts & Retries

```yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: ratings
spec:
  hosts:
    - ratings
  http:
  - route:
    - destination:
        host: ratings
        subset: v1
    retries:
      attempts: 3
      perTryTimeout: 2s
```

- Set retries

Testing Services

```yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: ratings
spec:
  hosts:
  - ratings
  http:
  - fault:
      delay:
        percent: 10
        fixedDelay: 5s
    route:
    - destination:
        host: ratings
        subset: v1
```

Testing Services

```yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: ratings
spec:
  hosts:
  - ratings
  http:
  - fault:
      delay:
        percent: 10
        fixedDelay: 5s
    route:
    - destination:
        host: ratings
        subset: v1
```

- Inject fault (delay 10% by 5 secs)

Testing Services

```yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: ratings
spec:
  hosts:
  - ratings
  http:
  - fault:
      abort:
        percent: 10
        httpStatus: 400
    route:
    - destination:
        host: ratings
        subset: v1
```

Testing Services

```yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: ratings
spec:
  hosts:
  - ratings
  http:
  - fault:
      abort:
        percent: 10
        httpStatus: 400
    route:
    - destination:
        host: ratings
        subset: v1
```

- Inject fault (400 error code 10%)

Testing Services

```yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: ratings
spec:
  hosts:
  - ratings
  http:
  - match:
    - sourceLabels:
        app: reviews
        version: v2
    fault:
      delay:
        fixedDelay: 5s
      abort:
        percent: 10
        httpStatus: 400
    route:
    - destination:
        host: ratings
        subset: v1
```

Testing Services

```yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: ratings
spec:
  hosts:
  - ratings
  http:
  - match:
    - sourceLabels:
        app: reviews
        version: v2
    fault:
      delay:
        fixedDelay: 5s
      abort:
        percent: 10
        httpStatus: 400
    route:
    - destination:
        host: ratings
        subset: v1
```

- Use delay and abort together to confirm services are resilient

Rule Priority

```yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
  - reviews
  http:
  - match:
    - headers:
        Foo:
          exact: bar
    route:
    - destination:
        host: reviews
        subset: v2
  - route:
    - destination:
        host: reviews
        subset: v1
```

- Read from top to bottom
- Once match found all other rules are ignored
- Best practice: Higher priority rules first

Rule Priority

```yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
  - reviews
  http:
  - match:
    - headers:
        Foo:
          exact: bar
    route:
    - destination:
        host: reviews
        subset: v2
  - route:
    - destination:
        host: reviews
        subset: v1
```

- Checks for Foo = bar, if match sends to v2
- If no match sends to v1

Traffic Management - DestinationRule

configures the set of policies to be applied to a request after VirtualService routing has occurred.

- authored by service owners
- circuit breaker
- load balancer settings
- TLS settings

DestinationRule

```yaml
apiVersion: networking.istio.io/v1alpha3
kind: DestinationRule
metadata:
  name: reviews
spec:
  host: reviews
  trafficPolicy:
    loadBalancer:
      simple: RANDOM
  subsets:
  - name: v1
    labels:
      version: v1
  - name: v2
    labels:
      version: v2
    trafficPolicy:
      loadBalancer:
        simple: ROUND_ROBIN
  - name: v3
    labels:
      version: v3
```

DestinationRule

```yaml
apiVersion: networking.istio.io/v1alpha3
kind: DestinationRule
metadata:
  name: reviews
spec:
  host: reviews
  trafficPolicy:
    loadBalancer:
      simple: RANDOM
  subsets:
  - name: v1
    labels:
      version: v1
  - name: v2
    labels:
      version: v2
    trafficPolicy:
      loadBalancer:
        simple: ROUND_ROBIN
  - name: v3
    labels:
      version: v3
```

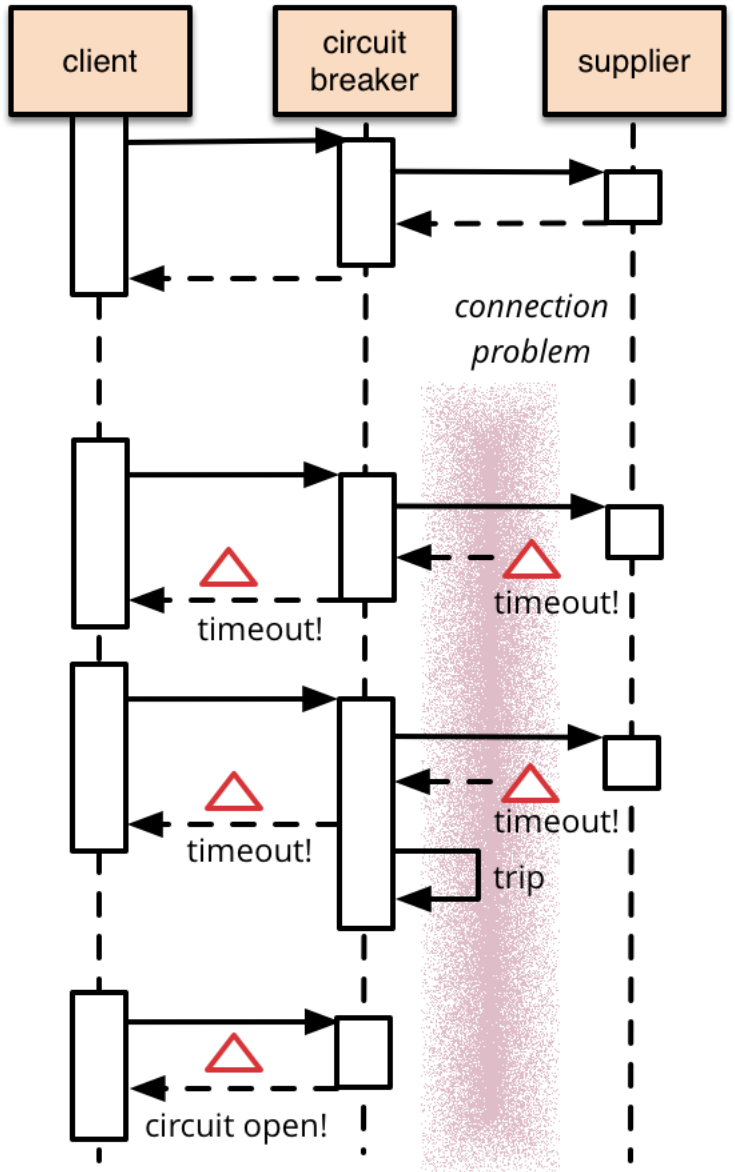
trafficPolicy (LB RANDOM)
trafficPolicy (LB Round_ROBIN

Circuit breakers

- critical component of distributed systems, which failing quickly, applying back pressure to downstream services

Circuit breakers

- External service calls (HTTP timeout)
- Cascading failures



Circuit breakers

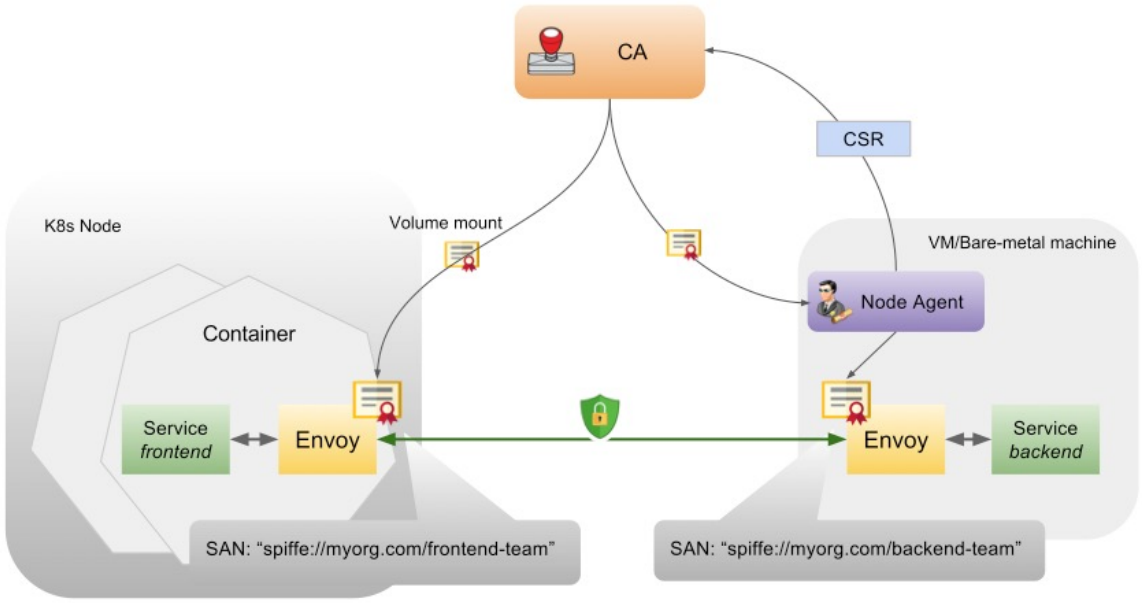
- External service calls (HTTP timeout)
- Cascading failures

```yaml
apiVersion: networking.istio.io/v1alpha3
kind: DestinationRule
metadata:
  name: reviews
spec:
  host: reviews
  subsets:
  - name: v1
    labels:
      version: v1
    trafficPolicy:
      connectionPool:
        tcp:
          maxConnections: 100
```

Istio: Fault injection

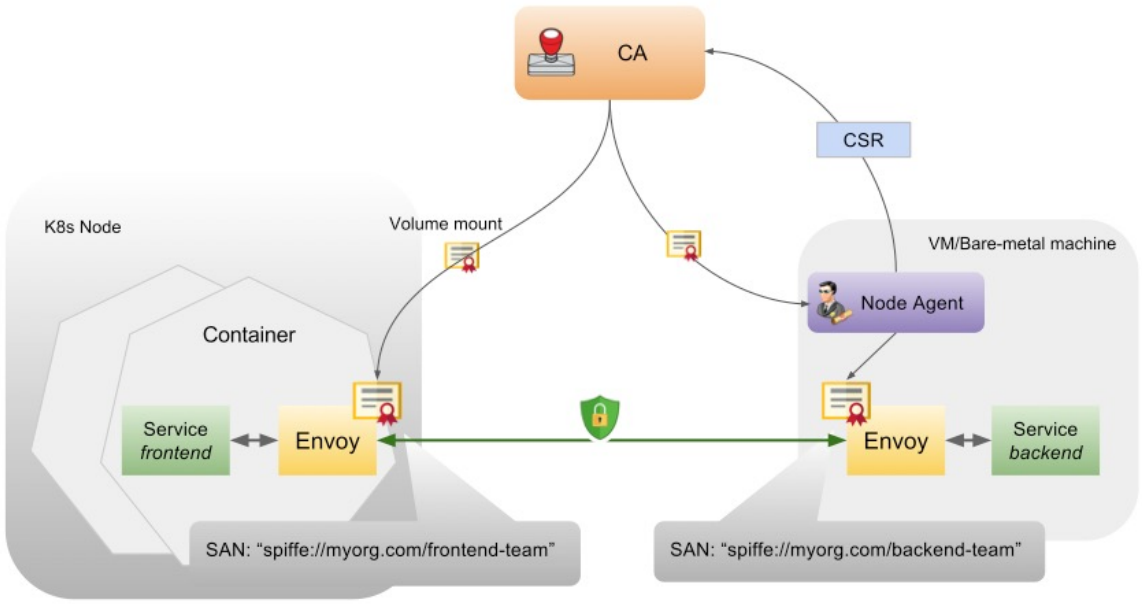
Istiod - Auth

- Identity
- Key management
- communication security
- Leverages secret volumes to deliver keys/certs from Istio CA to Kubernetes containers.
- Optional mutual TLS encryption



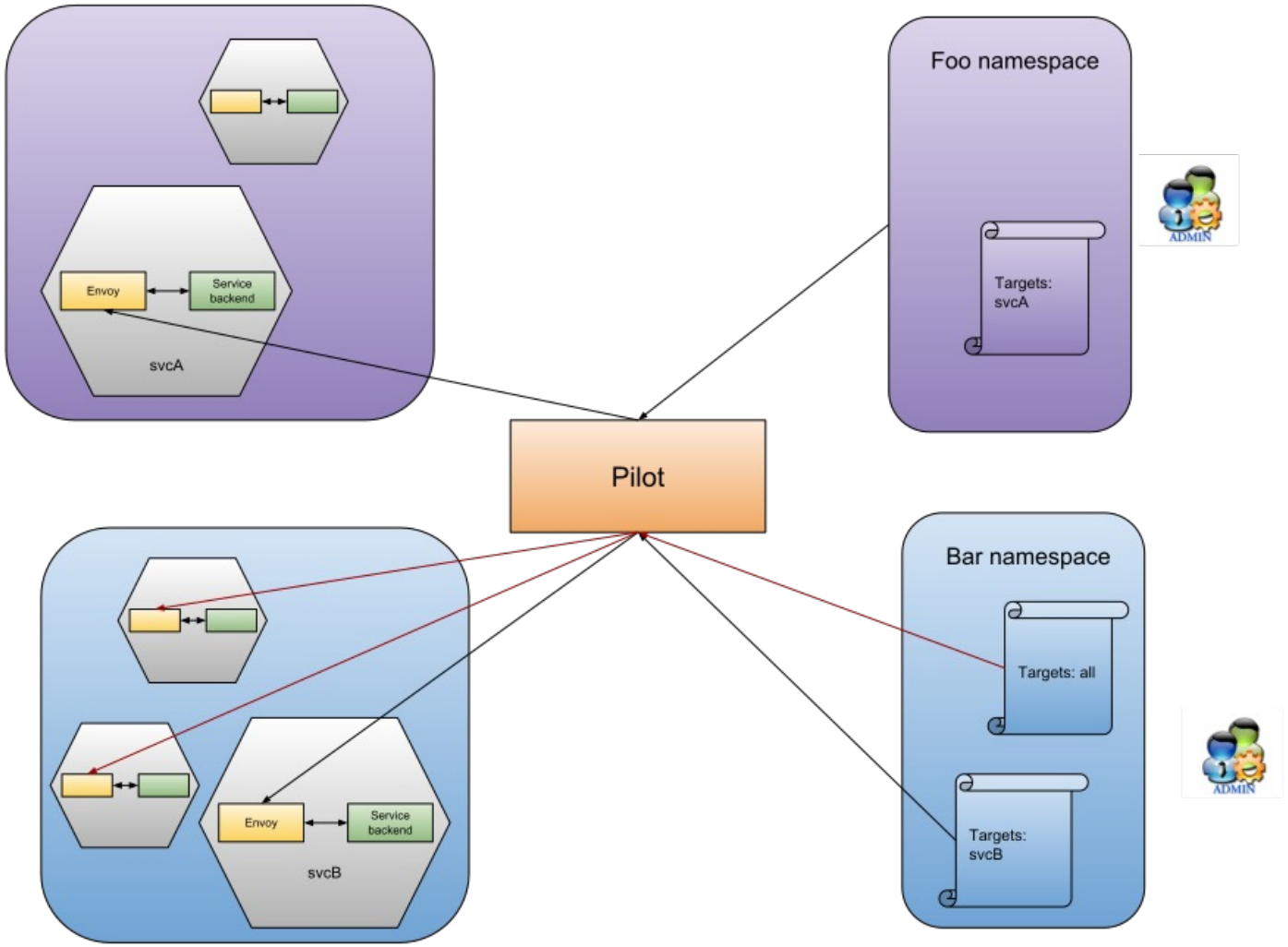
Istiod - Auth

- Automate key & cert management process:
  - generate key/cert pair for each service account
  - distribute key/cert pair to each pod
  - rotate keys/certs periodically
  - revoke key/cert pair when needed



Authentication Policy

- Istio authentication policy is composed of two parts:
- Peer: verifies the party, the direct client, that makes the connection. The common authentication mechanism for this is mutual TLS.
- Origin: verifies the party, the original client, that makes the request (e.g end-users, devices etc). JWT is the only supported mechanism for origin authentication at the moment.



Anatomy of policy

```
targets:
- name: product-page
- name: reviews
  ports:
  - number: 9000
```

- Target selectors
  - determine which hosts/ports policies applied to.

Enable mTLS

```yaml
apiVersion: "authentication.istio.io/v1alpha1"
kind: "Policy"
metadata:
  name: "example-2"
spec:
  targets:
  - name: httpbin
  peers:
  - mtls:
```

- Enable mTLS

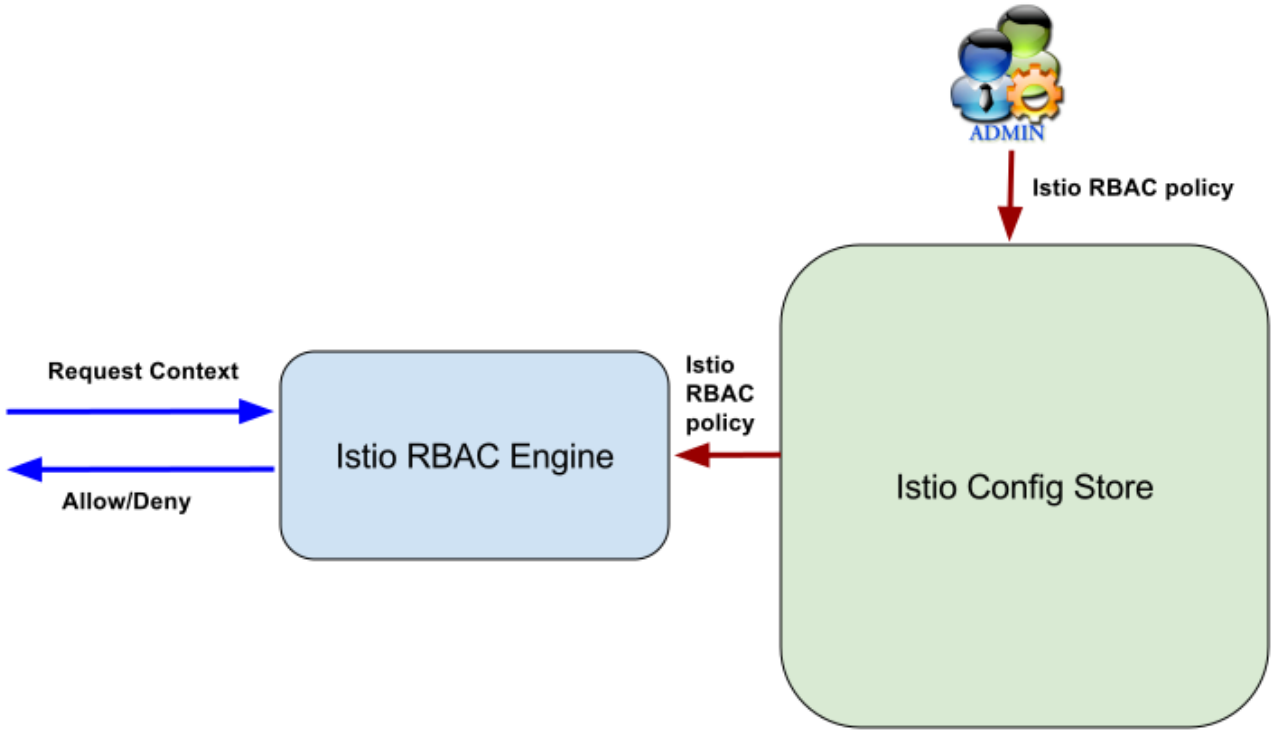
mTLS host

```yaml
apiVersion: "networking.istio.io/v1alpha3"
kind: "DestinationRule"
metadata:
  name: "example-2"
spec:
  host: "httpbin.bar.svc.cluster.local"
  trafficPolicy:
    tls:
      mode: ISTIO_MUTUAL
```

- Enable mTLS on httpbin host

Istio RBAC

- More granular
- Service to service
- end user to service



Istio RBAC

```
apiVersion: "config.istio.io/v1alpha2"
kind: authorization
metadata:
  name: requestcontext
  namespace: istio-system
spec:
  subject:
    user: source.user | ""
    groups: ""
    properties:
      service: source.service | ""
      namespace: source.namespace | ""
  action:
    namespace: destination.namespace | ""
    service: destination.service | ""
    method: request.method | ""
    path: request.path | ""
    properties:
      version: request.headers["version"] | ""
```

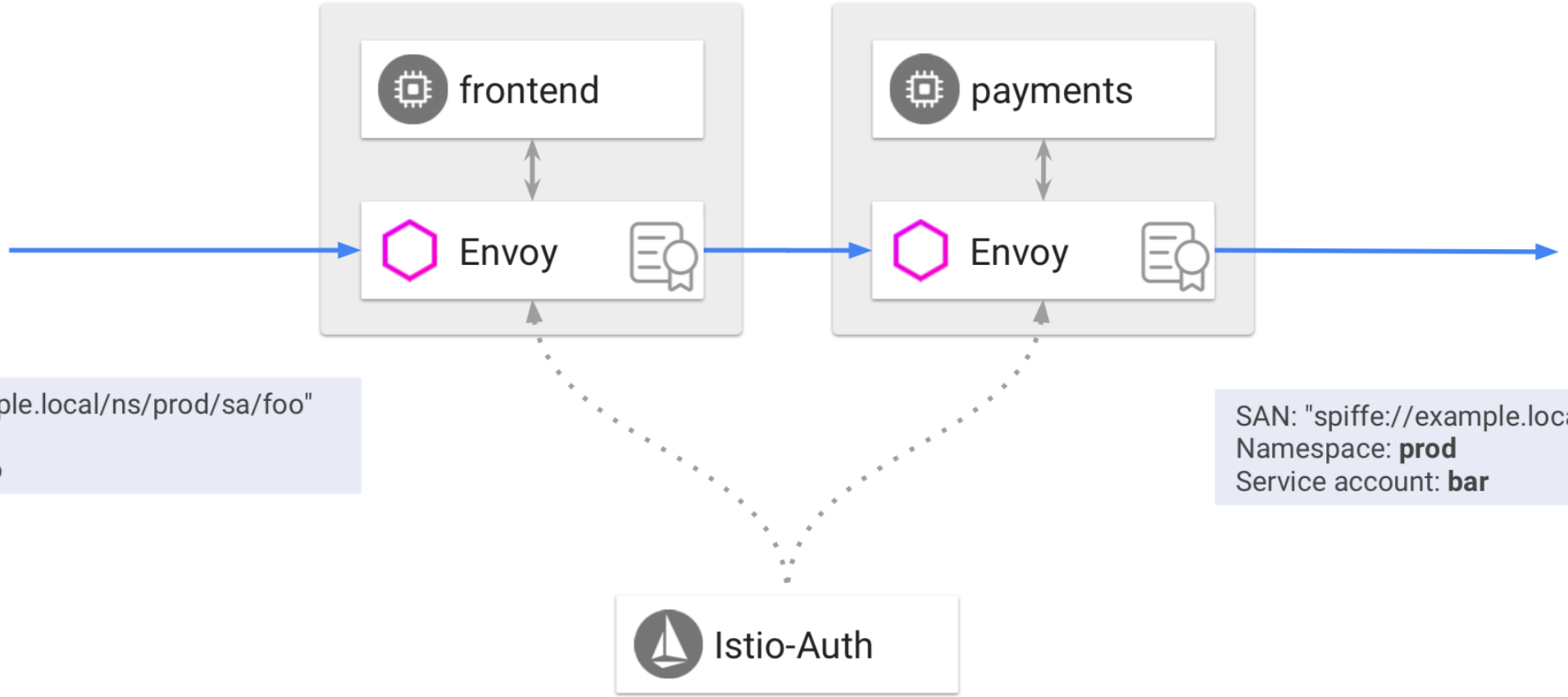
Istio RBAC

```yaml
apiVersion: "config.istio.io/v1alpha2"
kind: ServiceRole
metadata:
  name: products-viewer
  namespace: default
spec:
  rules:
  - services: ["products.default.svc.cluster.local"]
    methods: ["GET", "HEAD"]
```

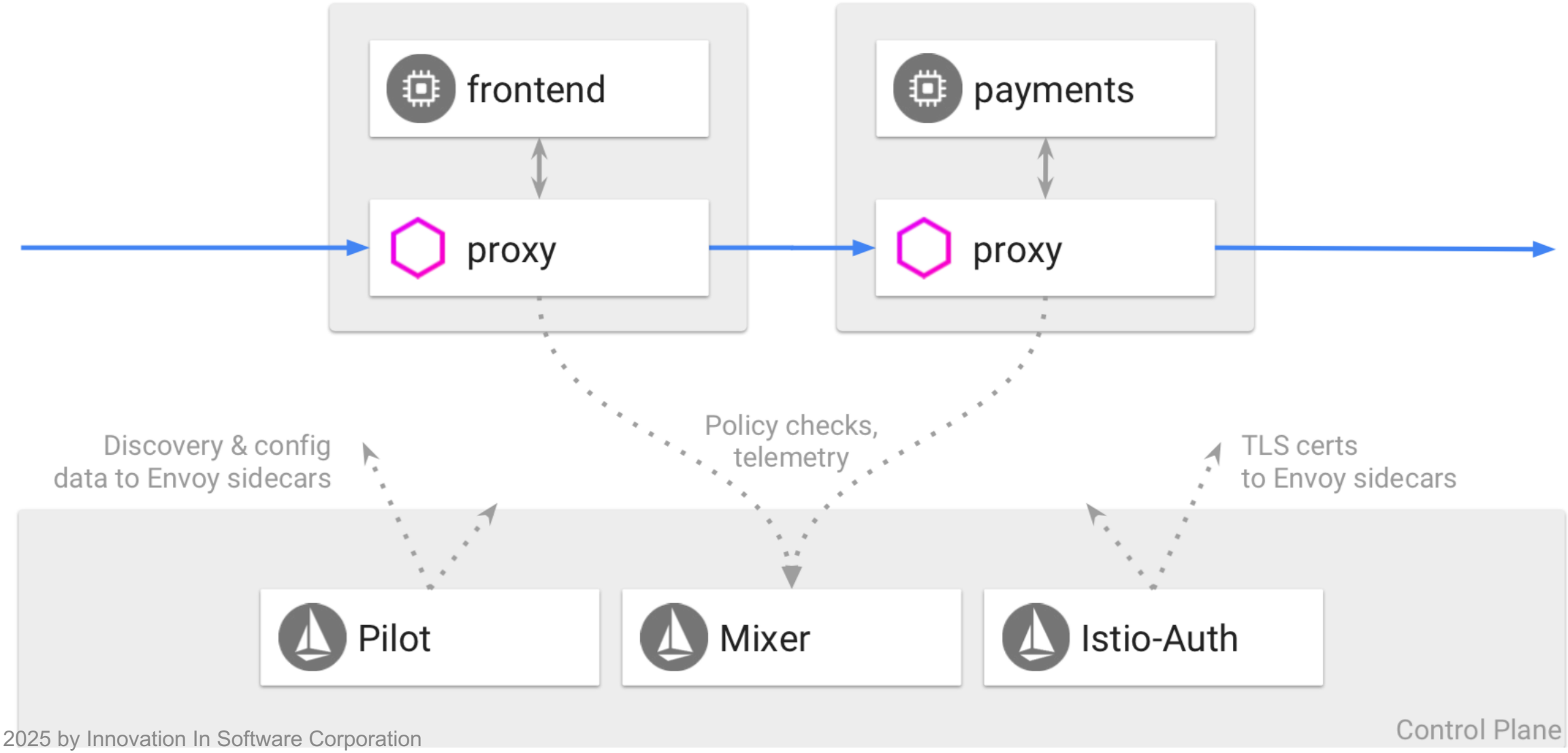
Istio RBAC

```yaml
apiVersion: "config.istio.io/v1alpha2"
kind: ServiceRole
metadata:
  name: products-viewer-version
  namespace: default
spec:
  rules:
  - services: ["products.default.svc.cluster.local"]
    methods: ["GET", "HEAD"]
    constraints:
    - key: "version"
      values: ["v1", "v2"]
```

Security



Security



Service Entries

- Istio blocks all outgoing external requests by default
- ServiceEntry required to allow connections to outside services
- mutal TLS authentication disabled
- Policy enforcement performed on client-side

ServiceEntry

```yaml
apiVersion: networking.istio.io/v1alpha3
kind: ServiceEntry
metadata:
  name: foo-ext-svc
spec:
  hosts:
  - *.foo.com
  ports:
  - number: 80
    name: http
    protocol: HTTP
  - number: 443
    name: https
    protocol: HTTPS
```

- Allow outgoing on HTTP & HTTPS to *.foo.com

VirtualService & ServiceEntry

```yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: bar-foo-ext-svc
spec:
  hosts:
    - bar.foo.com
  http:
  - route:
    - destination:
        host: bar.foo.com
    timeout: 10s
```

- Work with ServiceEntry to set a 10s timeout on external service

Gateways

- Describes load balancer at the edge of mesh
  - exposed ports
  - protocol
  - TLS

Gateway

```yaml
apiVersion: networking.istio.io/v1alpha3
kind: Gateway
metadata:
  name: my-gateway
spec:
  selector:
    app: my-gatweway-controller
  servers:
  - port:
      number: 80
      name: http
      protocol: HTTP
    hosts:
    - uk.bookinfo.com
    - eu.bookinfo.com
    tls:
      httpsRedirect: true # sends 302 redirect for http requests
  - port:
      number: 443
      name: https
      protocol: HTTPS
```

Gateway

```
  hosts:
  - uk.bookinfo.com
  - eu.bookinfo.com
  tls:
    mode: SIMPLE #enables HTTPS on this port
    serverCertificate: /etc/certs/servercert.pem
    privateKey: /etc/certs/privatekey.pem
- port:
    number: 9080
    name: http-wildcard
    protocol: HTTP
  hosts:
  - "*"
- port:
    number: 2379 # to expose internal service via external port 2379
    name: mongo
    protocol: MONGO
  hosts:
  - "*"
```

Gateway/VirtualService

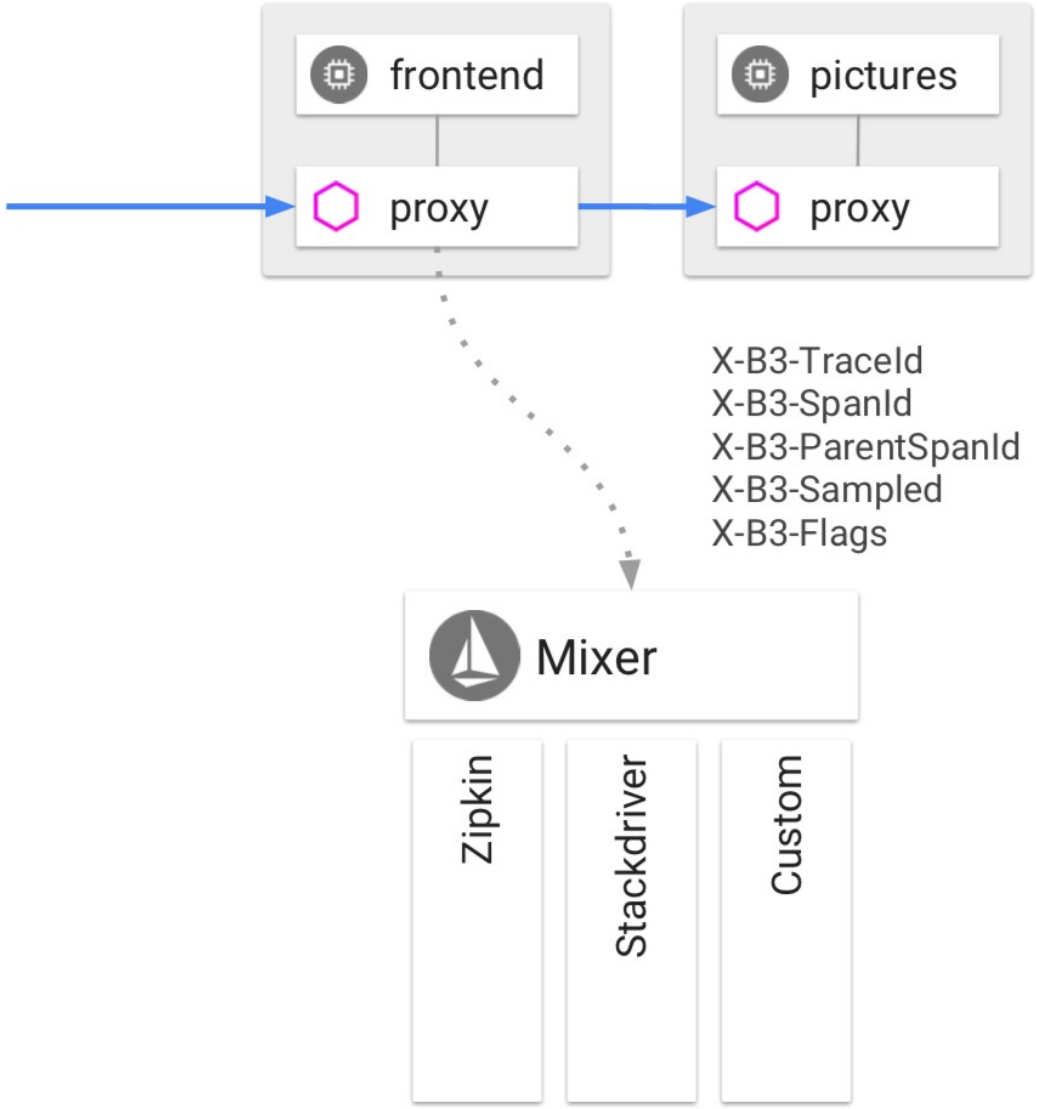
```yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: bookinfo
spec:
  hosts:
    - bookinfo.com
  gateways:
  - bookinfo-gateway # <---- bind to gateway
  http:
  - match:
    - uri:
        prefix: /reviews
    route:
```

Visibility

- Metrics without code changes
- Consistent metrics for every app deployed
- Trace flow of requests across services
- Portable across metric backend providers

Tracing

- Apps do not have to deal with generating spans or correlating causality
- Envoy proxies generate spans
  - Apps do need to send some context heads on outbound calls
- Envoys send traces to Mixer
  - Adapters send to backends



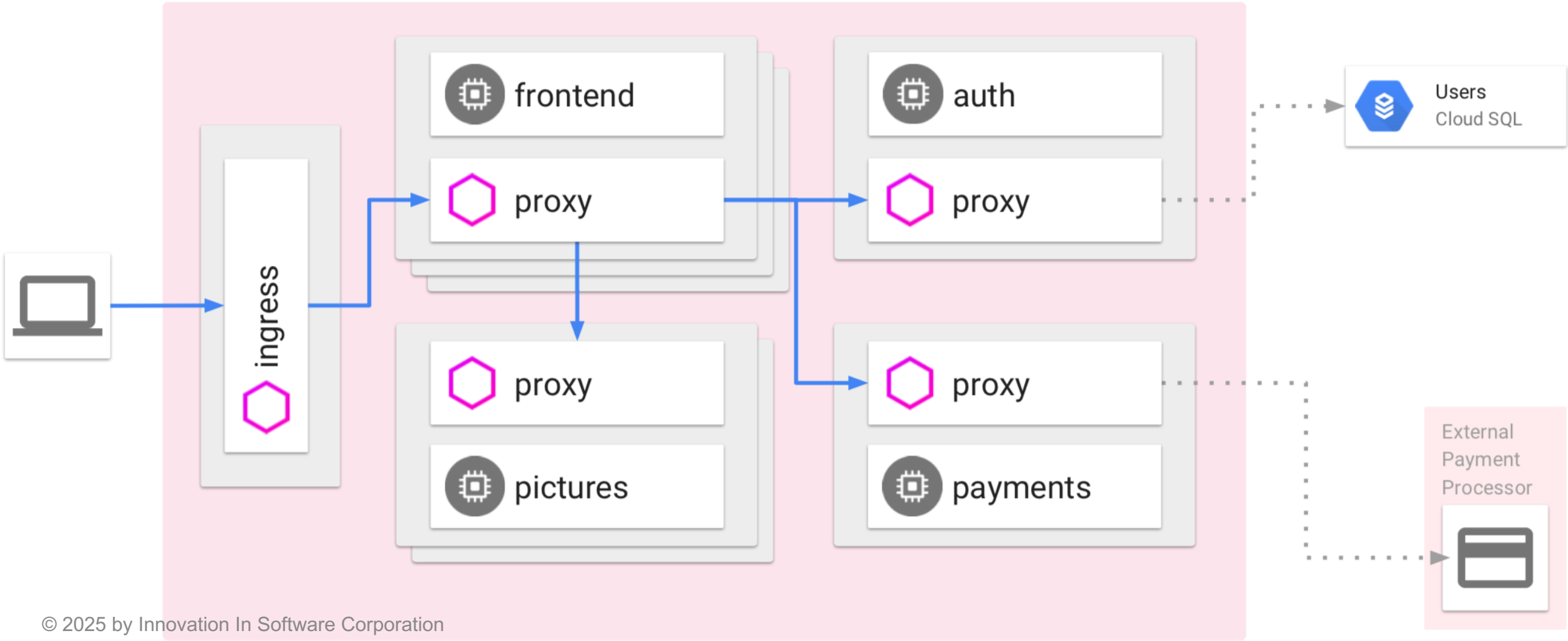
Tracing - headers

```
def getForwardHeaders(request):
    headers = {}

    user_cookie = request.cookies.get("user")
    if user_cookie:
        headers['Cookie'] = 'user=' + user_cookie

    incoming_headers = [ 'x-request-id',
                         'x-b3-traceid',
                         'x-b3-spanid',
                         'x-b3-parentspanid',
                         'x-b3-sampled',
                         'x-b3-flags',
                         'x-ot-span-context'
    ]

    for ihdr in incoming_headers:
        val = request.headers.get(ihdr)
        if val is not None:
            headers[ihdr] = val
```

Tracing - headers

```
@GET
@Path("/reviews")
public Response bookReviews(@CookieParam("user") Cookie user,
                            @HeaderParam("x-request-id") String xreq,
                            @HeaderParam("x-b3-traceid") String xtraceid,
                            @HeaderParam("x-b3-spanid") String xspanid,
                            @HeaderParam("x-b3-parentspanid") String xparentspanid,
                            @HeaderParam("x-b3-sampled") String xsampled,
                            @HeaderParam("x-b3-flags") String xflags,
                            @HeaderParam("x-ot-span-context") String xotspan) {
  String r1 = "";
  String r2 = "";

  if(ratings_enabled){
    JsonObject ratings = getRatings(user, xreq, xtraceid, xspanid, xparentspanid, xsampled, xflags, xotspan
```

Service Mesh



Ultimately, it's just this